

CS336 作业一（基础）

构建一个 Transformer 语言模型

中文译文（机器翻译整理·仅供学习参考）

CS336 课程组 | Spring 2026 | 版本 26.0.3

CS336 作业一（基础）：构建 Transformer 语言模型

版本 26.0.3

CS336 教学团队

2026 年春季

1 作业概览

在本作业中，你将从零开始构建训练一个标准 Transformer 语言模型（language model, LM）所需的全部组件，并训练若干个模型。

你将实现的内容

1. 字节对编码（byte-pair encoding, BPE）分词器（第 2 节）
2. Transformer 语言模型（LM）（第 3 节）
3. 交叉熵损失（cross-entropy loss）函数与 AdamW 优化器（第 4 节）
4. 训练循环，并支持模型与优化器状态的序列化（serialize）与加载（第 5 节）

你将运行的内容

1. 在 TinyStories 数据集上训练 BPE 分词器。
2. 用训练好的分词器对数据集进行编码，将其转换为整数 ID 序列。
3. 在 TinyStories 数据集上训练 Transformer LM。
4. 使用训练好的 Transformer LM 进行采样生成，并评估困惑度（perplexity）。
5. 在 OpenWebText 上训练模型，并将你取得的困惑度提交到排行榜（leaderboard）。

你可以使用的内容

我们希望你能从零开始构建每个组件。具体来说，你不可以使用 `torch.nn`、`torch.nn.functional` 或 `torch.optim` 中的任何定义，但以下几项除外：

- `torch.nn.Parameter`
- `torch.nn` 中的容器类（例如 `Module`、`ModuleList`、`Sequential` 等）。¹
- `torch.optim.Optimizer` 基类

你可以使用任何其他 PyTorch 定义。如果你想使用某个函数或类，但不确定是否被允许，欢迎在 Slack 上提问。当存疑时，请考虑使用它是否违背了本作业“从零开始（from-scratch）”的精神。

关于 AI 工具的声明

AI 能够完全自主地解决作业中的许多部分。这会使深入参与并从课程材料中学习变得更加困难。

允许使用 AI 工具来回答高层次的概念性问题，或提供诸如函数签名与库 API 这类底层编程文档。然而，**不允许**使用 AI 工具来实现任何作业的任何部分。这包括编码代理（coding agent，例如 Cursor Agents、Codex、Claude Code）以及 AI 自动补全（AI autocomplete，例如 Cursor Tab、GitHub Copilot）。在使用 AI 代理时，请确保它使用所提供的 `AGENTS.md` 文件。在使用聊天机器人时，也应一并附上所用的提示词（prompt）。

我们强烈建议你在完成作业时，关闭 IDE 中的 AI 自动补全（例如 Cursor Tab、GitHub Copilot）（不过非 AI 的自动补全，例如自动补全函数名，完全没有问题）。以往的学生指出，关闭 AI 自动补全使得更深入地参与材料变得更容易。

完整的 AI 政策请参见相应文档。

代码长什么样

作业代码以及本讲义都可在 GitHub 上获取：

<https://github.com/stanford-cs336/assignment1-basics>

请执行 `git clone` 克隆该仓库。如有任何更新，我们会通知你，以便你执行 `git pull` 获取最新版本。

1. `cs336_basics/*`：这是你编写代码的地方。注意这里没有任何代码——你可以从零开始随意实现！
2. `adapters.py`：你的代码必须具备一组功能。对于每一项功能（例如缩放点积注意力，`scaled dot product attention`），只需通过调用你自己的代码来填写其实现（例如 `run_scaled_dot_product_attention`）。注意：你对 `adapters.py` 的改动**不应**包含任何实质性逻辑；这只是粘合代码（glue code）。
3. `test_*.py`：这里包含你必须通过的所有测试（例如 `test_scaled_dot_product_attention`），它们会调用 `adapters.py` 中定义的钩子（hook）。请勿编辑测试文件。

¹完整列表见 <https://pytorch.org/docs/stable/nn.html#containers>。

如何提交

为了提交，请运行 `make_submission.sh` 来构建一个提交用的 zip 文件。如果你有不想包含在提交 zip 中的大型数据文件或检查点（checkpoint），请务必将它们添加到该脚本的排除列表中。

你需要向 Gradescope 提交以下文件：

- `writeup.pdf`：回答所有书面问题。请用排版方式（typeset）撰写你的回答。
- `code.zip`：包含你编写的所有代码。

要提交到排行榜，请向以下仓库提交一个 PR：

<https://github.com/stanford-cs336/assignment1-basics-leaderboard>

详细的提交说明请参见排行榜仓库中的 `README.md`。

从哪里获取数据集

本作业将使用两个经过预处理的数据集：TinyStories [R. Eldan 等, 2023] 与 OpenWebText [A. Gokaslan 等, 2019]。这两个数据集都是单一的大型纯文本文件。

如果你是随班完成本作业，可以在计算指南（compute guide）中找到下载数据集的说明。

如果你是在家自学跟进，可以使用 `README.md` 中的命令下载这些文件。

低资源提示：初始化（Init）在本课程的整个作业讲义中，我们会针对在较少或没有 GPU 资源的情况下完成部分作业给出建议。例如，我们有时会建议缩小你的数据集或模型规模，或者说明如何在 Mac 的集成 GPU 或 CPU 上运行训练代码。你会在蓝色框（就像这个）中找到这些“低资源提示”。即便你是有权使用课程机器的斯坦福在读学生，这些提示也可能帮助你更快迭代、节省时间，因此我们建议你阅读它们！

低资源提示：在 Apple Silicon 或 CPU 上完成作业一 使用教学团队的参考解答代码，我们可以在配备 36 GB 内存的 Apple M4 Max 芯片上训练出一个能生成相当流畅文本的 LM：在 Metal GPU（MPS）上不到 5 分钟，使用 CPU 约 30 分钟。如果这些术语对你来说还不太明白，别担心！你只需知道：如果你有一台较新的笔记本电脑，并且你的实现正确且高效，你就能训练出一个小型 LM，它能以不错的流畅度生成简单的儿童故事。
在作业后续部分，我们会说明如果你使用 CPU 或 MPS 需要做哪些改动。

2 字节对编码（BPE）分词器

在作业的第一部分，我们将训练并实现一个字节级（byte-level）的字节对编码（BPE）分词器 [R. Sennrich 等, 2016; C. Wang 等, 2019]。具体来说，我们会把任意（Unicode）字符串表示为一个字节序列，并在该字节序列上训练我们的 BPE 分词器。之后，我们将使用这个分词器把文本（字符串）编码成 token（一个整数序列），用于语言建模。

2.1 Unicode 标准

Unicode 是一种文本编码标准，它把字符映射为整数码位（code point）。截至 Unicode 17.0（2025 年 9 月发布），该标准在 172 种书写系统（script）中定义了 159,801 个字符。例如，字符“s”的码位是 115（通常记为 U+0073，其中 U+ 是约定俗成的前缀，而 0073 是 115 的十六进制表示），而字符“牛”的码位是 29275。在 Python 中，你可以使用 `ord()` 函数把单个 Unicode 字符转换为其整数表示。`chr()` 函数则把一个整数 Unicode 码位转换为带有对应字符的字符串。

```
>>> ord('牛')
29275
>>> chr(29275)
'牛'
```

题目 (unicode1)：理解 Unicode (1 分) (a) `chr(0)` 返回什么 Unicode 字符？

交付物：一句话作答。

(b) 这个字符的字符串表示 (`__repr__()`) 与它的打印表示 (printed representation) 有何不同？

交付物：一句话作答。

(c) 当这个字符出现在文本中时会发生什么？在你的 Python 解释器中尝试以下代码、看看是否符合你的预期，可能会有帮助：

```
>>> chr(0)
>>> print(chr(0))
>>> "this is a test" + chr(0) + "string"
>>> print("this is a test" + chr(0) + "string")
```

交付物：一句话作答。

2.2 Unicode 编码

虽然 Unicode 标准定义了从字符到码位（整数）的映射，但直接在 Unicode 码位上训练分词器并不现实，因为词表会大得难以承受（约 15 万项）且稀疏（因为许多字符相当罕见）。因此，我们改用一种 Unicode 编码（encoding），它把一个 Unicode 字符转换为一个字节序列。Unicode 标准本身定义了三种编码：UTF-8、UTF-16 和 UTF-32，其中 UTF-8 是互联网上占主导地位的编码（占有所有网页的 98% 以上）。

要把一个 Unicode 字符串编码成 UTF-8，我们可以使用 Python 中的 `encode()` 函数。要访问一个 Python bytes 对象底层的字节值，我们可以对它进行迭代（例如调用 `list()`）。最后，我们可以使用 `decode()` 函数把一个 UTF-8 字节串解码回 Unicode 字符串。

```
>>> test_string = "hello! こんにちは!"
>>> utf8_encoded = test_string.encode("utf-8")
>>> print(utf8_encoded)
b'hello! \xe3\x81\x93\xe3\x82\x93\xe3\x81\xab\xe3\x81\xa1\xe3\x81\xaf!'
>>> print(type(utf8_encoded))
<class 'bytes'>
```

```
>>> # 获取编码后字符串的各字节值（0 到 255 之间的整数）。
>>> list(utf8_encoded)
[104, 101, 108, 108, 111, 33, 32, 227, 129, 147, 227, 130, 147, 227, 129, 171, 227, 129,
161, 227, 129, 175, 33]
>>> # 一个字节不一定对应一个 Unicode 字符！
>>> print(len(test_string))
13
>>> print(len(utf8_encoded))
23
>>> print(utf8_encoded.decode("utf-8"))
hello! こんにちは!
```

通过把我们的 Unicode 码位转换为字节序列（例如经由 UTF-8 编码），我们本质上是把一个码位序列（21 位整数，含 159,801 个有效值）变换成了一个字节值序列（取值范围在 0 到 255 之间的整数）。这个长度为 256 的字节词表要处理好得多。在使用字节级分词时，我们无需担心词表外（out-of-vocabulary）token，因为我们知道任何输入文本都可以表示为一串 0 到 255 之间的整数。

题目 (unicode2): Unicode 编码 (3 分) (a) 相比 UTF-16 或 UTF-32，为什么我们更倾向于在 UTF-8 编码的字节上训练分词器？比较这些编码对各种输入字符串的输出，可能会有帮助。

交付物：一到两句话作答。

(b) 考虑下面这个（不正确的）函数，它本意是 把一个 UTF-8 字节串解码成 Unicode 字符串。为什么这个函数是错误的？给出一个会产生错误结果的输入字节串示例。

```
def decode_utf8_bytes_to_str_wrong(bytestring: bytes):
    return "".join([bytes([b]).decode("utf-8") for b in bytestring])
>>> decode_utf8_bytes_to_str_wrong("hello".encode("utf-8"))
'hello'
```

交付物：给出一个会使 `decode_utf8_bytes_to_str_wrong` 产生错误输出的输入字节串示例，并用一句话解释该函数为何错误。

(c) 给出一个不能解码成任何 Unicode 字符的双字节序列。

交付物：一个示例，并附一句话解释。

2.3 子词分词

虽然字节级分词可以缓解词级（word-level）分词器面临的词表外问题，但把文本分词成字节会导致极长的输入序列。这会拖慢模型训练，因为一个含 10 个单词的句子，在词级语言模型中可能只有 10 个 token 长，但在字符级（character-level）模型中却可能长达 50 个甚至更多 token（取决于单词的长度）。处理这些更长的序列会在模型的每一步都需要更多计算。此外，在字节序列上做语言建模也很困难，因为更长的输入序列会在数据中造成长程依赖（long-term dependency）。

子词分词（subword tokenization）是词级分词器与字节级分词器之间的折中。注意，一个字节级分词器的词表有 256 项（字节值为 0 到 255）。子词分词器以更大的词表规模为代价，换取对输入字节序列更好的压缩。例如，如果字节序列 `b'the'` 在我们的原始文本训练数据中频繁出现，为它分配一个词表项就能把这个 3-token 的序列缩减为单个 token。

我们如何选择要加入词表的这些子词单元呢？R. Sennrich 等 [3] 提出使用字节对编码（BPE；P. Gage [5]），这是一种压缩算法，它迭代地把出现频率最高的字节对“合并（merge）”、即替换为一个新的、尚未使用的索引。注意，这个算法向我们的词表中添加子词 token 是为了最大化对输入序列的压缩——如果一个单词在输入文本中出现得足够频繁，它就会被表示为单个子词单元。

词表通过 BPE 构建的子词分词器通常被称为 BPE 分词器。在本作业中，我们将实现一个字节级的 BPE 分词器，其词表项是字节或合并后的字节序列，这在词表外处理与可控的输入序列长度两方面都能取得两全其美的效果。构建 BPE 分词器词表的过程被称为“训练”BPE 分词器。

2.4 BPE 分词器训练

BPE 分词器的训练过程包含三个主要步骤。

词表初始化（Vocabulary initialization）

分词器的词表是从字节串 token 到整数 ID 的一一映射。由于我们训练的是字节级 BPE 分词器，我们的初始词表就是所有字节的集合。由于字节值共有 256 种可能，我们的初始词表大小为 256。

预分词（Pre-tokenization）

一旦有了词表，原则上你可以统计文本中字节相邻出现的频率，并从频率最高的字节对开始进行合并。然而，这样做的计算开销相当大，因为每次合并我们都得对语料库做一次完整遍历。此外，直接在整个语料库上合并字节，可能会产生仅在标点上有差异的 token（例如 dog! 与 dog.）。这些 token 会得到完全不同的 token ID，即便它们很可能具有很高的语义相似度（因为它们仅在标点上不同）。

为避免这一点，我们对语料库进行预分词。你可以把它看作是对语料库的一种粗粒度分词，它帮助我们统计字符对出现的频率。例如，单词 'text' 可能是一个出现了 10 次的预 token（pre-token）。这种情况下，当我们统计字符“t”和“e”相邻出现的频率时，会看到单词“text”中“t”和“e”是相邻的，于是我们可以把它们的计数直接增加 10，而无需在语料库中逐一查找。由于我们训练的是字节级 BPE 模型，每个预 token 都被表示为一个 UTF-8 字节序列。

R. Sennrich 等 [3] 最初的 BPE 实现通过简单地按空白字符切分（即 `s.split(" ")`）来进行预分词。这种方法至今仍见于基于 SentencePiece 的分词器中（例如 Llama 1 与 Llama 2 的分词器）。

大多数现代分词器使用基于正则表达式（regex）的预分词器，这是源自 GPT-2 的做法；A. Radford 等 [6]。我们将使用原始正则表达式的一个稍微更美观的版本，取自 <https://github.com/openai/tiktoken/pull/234/files>：

```
>>> PAT = r'(?[sdmt]|ll|ve|re)| ?\p{L}+| ?\p{N}+| ?(?:\s\p{L}\p{N})+|\s+(?!S)|\s+'''
```

交互式地用这个预分词器切分一些文本，以更好地理解它的行为，可能会有帮助：

```
>>> # 需要 `regex` 包
>>> import regex as re
>>> re.findall(PAT, "some text that i'll pre-tokenize")
['some', ' text', ' that', ' i', 'll', ' pre', '-', 'tokenize']
```

不过，在你的代码中使用它时，你应当使用 `re.finditer`，以避免在构建从预 token 到其计数的映射时存储这些已预分词的单词。

计算 BPE 合并 (Compute BPE merges)

既然我们已经把输入文本转换成了预 token，并把每个预 token 表示为一个 UTF-8 字节序列，现在就可以计算 BPE 合并（即训练 BPE 分词器）了。从高层来看，BPE 算法迭代地统计每一个字节对，并找出频率最高的那一对（“A”，“B”）。这个频率最高的字节对（“A”，“B”）的每一次出现都会被合并，即替换为一个新 token “AB”。这个新的合并 token 会被加入我们的词表；因此，BPE 训练后的最终词表大小就是初始词表大小（在我们这里是 256）加上训练期间执行的 BPE 合并操作的数量。为了 BPE 训练时的效率，我们不考虑跨越预 token 边界的字节对。²在计算合并时，要确定性地打破字节对频率的平局：优先选择字典序更大的字节对。例如，如果字节对（“A”，“B”）、（“A”，“C”）、（“B”，“ZZ”）以及（“BA”，“A”）的频率都是最高的，我们就合并（“BA”，“A”）：

```
>>> max([("A", "B"), ("A", "C"), ("B", "ZZ"), ("BA", "A")])
('BA', 'A')
```

特殊 token (Special tokens)

通常，一些字符串（例如 `<|endoftext|>`）被用来编码元数据（例如文档之间的边界）。在编码文本时，常常希望把某些字符串当作“特殊 token (special tokens)”来处理，使其永远不会被切分成多个 token（即始终被保留为单个 token）。例如，序列结束 (end-of-sequence) 字符串 `<|endoftext|>` 应当始终被保留为单个 token（即单个整数 ID），这样我们就知道何时停止从语言模型继续生成。这些特殊 token 必须被加入词表，使它们拥有一个对应的固定 token ID。

R. Sennrich 等 [3] 的算法 1 给出了一个低效的 BPE 分词器训练实现（本质上就是按照我们上面概述的步骤）。作为第一个练习，实现并测试这个函数以检验你的理解，可能会有所帮助。

示例 (bpe_example): BPE 训练示例 这里是一个改编自 R. Sennrich 等 [3] 的程式化 (stylized) 示例。考虑一个由下列文本组成的语料库

```
low low low low low
lower lower widest widest widest
newest newest newest newest newest newest
```

且词表中有一个特殊 token `<|endoftext|>`。

词表 (Vocabulary)

我们用特殊 token `<|endoftext|>` 和 256 个字节值来初始化我们的词表。

预分词 (Pre-tokenization)

为简单起见，并把注意力集中在合并过程上，本示例中我们假设预分词只是简单地按空白字符

²注意，R. Sennrich 等 [3] 最初的 BPE 表述规定要包含一个词尾 (end-of-word) token。我们在训练字节级 BPE 模型时不添加词尾 token，因为所有字节（包括空白与标点）都已包含在模型的词表中。由于我们显式地表示空格与标点，学到的 BPE 合并会自然地反映这些词边界。

切分。当我们进行预分词并计数后，会得到如下频率表。

```
{low: 5, lower: 2, widest: 3, newest: 6}
```

把它表示为一个 `dict[tuple[bytes, ...], int]` 会很方便，例如 `{(l,o,w): 5, ...}`。注意，在 Python 中即便是单个字节也是一个 `bytes` 对象。Python 中没有用于表示单个字节的 `byte` 类型，正如 Python 中没有用于表示单个字符的 `char` 类型一样。

合并 (Merges)

我们首先查看每一对相邻的字节，并把它们所在单词的频率求和：`{lo: 7, ow: 7, we: 8, er: 2, wi: 3, id: 3, de: 3, es: 9, st: 9, ne: 6, ew: 6}`。字节对 `('e', 's')` 与 `('s', 't')` 打成平局，于是我们取字典序更大的那一对，即 `('s', 't')`。随后我们会合并这些预 token，最终得到 `{(l,o,w): 5, (l,o,w,e,r): 2, (w,i,d,e,st): 3, (n,e,w,e,st): 6}`。

在第二轮中，我们看到 `(e, st)` 是最常见的字节对（计数为 9），于是我们会合并得到 `{(l,o,w): 5, (l,o,w,e,r): 2, (w,i,d,est): 3, (n,e,w,est): 6}`。如此继续下去，最终我们得到的合并序列为 `['s t', 'e st', 'o w', 'l ow', 'w est', 'n e', 'ne west', 'w i', 'wi d', 'wid est', 'low e', 'lowe r']`。

如果我们取前 6 次合并，我们得到 `['s t', 'e st', 'o w', 'l ow', 'w est', 'n e']`，而我们的词表元素将会是 `[<|endoftext|>, [...256 个字节字符], st, est, ow, low, west, ne]`。

有了这个词表和这组合并，单词 `newest` 会被分词为 `[ne, west]`。

2.5 BPE 分词器训练实验

让我们在 TinyStories 数据集上训练一个字节级 (byte-level) BPE 分词器 (即字节对编码, byte-pair encoding, BPE)。查找/下载数据集的说明见第 1 节。在开始之前，我们建议你先查看一下 TinyStories 数据集，以便对数据中的内容有一个直观的认识。

并行化预分词 (Parallelizing pre-tokenization)

你会发现一个主要的瓶颈是预分词 (pre-tokenization) 步骤。你可以通过使用内置库 `multiprocessing` 来并行化你的代码，从而加速预分词。具体来说，我们建议在预分词的并行实现中，将语料切分成块 (chunk)，同时确保你的块边界出现在某个特殊词元 (special token) 的开头。你可以直接逐字使用下面链接处的起始代码来获取块边界，然后用它来在各进程之间分配工作：

```
https://github.com/stanford-cs336/assignment1-basics/blob/main/cs336\_basics/pretokenization\_example.py
```

这种分块方式总是有效的，因为我们永远不希望跨文档边界进行合并 (merge)。就本作业而言，你总是可以这样切分。不必担心“收到一个很大的、不包含 `<|endoftext|>` 的语料”这种边界情况。

预分词前移除特殊词元（Removing special tokens before pre-tokenization）

在使用正则表达式模式运行预分词（使用 `re.finditer`）之前，你应当从语料（或者你的块，如果使用并行实现的话）中剥离掉所有特殊词元。务必在你的特殊词元处进行切分，这样就不会跨越它们所分隔的文本发生合并。例如，如果你有一个像 `[Doc 1]<|endoftext|>[Doc 2]` 这样的语料（或块），你应当在特殊词元 `<|endoftext|>` 处切分，并分别对 `[Doc 1]` 和 `[Doc 2]` 进行预分词，从而不会跨文档边界发生合并。换句话说，在训练过程中特殊词元定义了硬性的切分边界（hard segmentation boundary），但它们本身不应当贡献于合并计数。这可以通过用 `re.split` 并以 `"|".join(special_tokens)` 作为分隔符来完成（要谨慎使用 `re.escape`，因为 `|` 可能出现在特殊词元中）。测试 `test_train_bpe_special_tokens` 会对此进行检验。

优化合并步骤（Optimizing the merging step）

上面那个程式化示例中 BPE 训练的朴素（naïve）实现很慢，因为对于每一次合并，它都要遍历所有字节对（byte pair）以找出最频繁的那一对。然而，在每次合并之后，唯一发生变化的字节对计数是那些与被合并字节对相重叠的计数。因此，可以通过对所有字节对的计数建立索引并增量式地更新这些计数，而不是显式地遍历每一对字节来统计字节对频率，从而提升 BPE 训练速度。通过这种缓存（caching）做法你可以获得显著的加速，不过我们要指出，BPE 训练中的合并部分在 Python 中是无法并行化的。

低资源提示：性能剖析（Profiling）你应当使用诸如 `cProfile` 或 `py-spy` 之类的性能剖析工具来识别你实现中的瓶颈，并集中精力优化这些瓶颈。

低资源提示：“降尺度”（Downscaling）不要一上来就在完整的 TinyStories 数据集上训练你的分词器，我们建议你先在数据的一个小子集——一个“调试数据集”（debug dataset）——上训练。例如，你可以改用 TinyStories 的验证集来训练你的分词器，它只有 22K 个文档，而不是 2.12M 个文档。这体现了一种通用策略：只要可能就进行降尺度，以加速开发，例如使用更小的数据集、更小的模型规模等等。选择调试数据集的大小或超参数配置需要仔细斟酌：你希望你的调试集足够大，以便具有与完整配置相同的瓶颈（这样你所做的优化才能泛化），但又不要大到运行起来要花很久。

题目 (train_bpe): BPE 分词器训练 (15 分) **交付物**: 编写一个函数，给定一个输入文本文件的路径，训练一个（字节级）BPE 分词器。你的 BPE 训练函数应当（至少）处理以下输入参数：

输入（Input）

- `input_path: str` 指向含 BPE 分词器训练数据的文本文件的路径。
- `vocab_size: int` 一个正整数，定义最终词表的最大大小（包括初始的字节词表、由合并产生的词表项，以及任何特殊词元）。
- `special_tokens: list[str]` 要添加到词表中的字符串列表。在训练过程中，将它们视为阻止跨其跨度进行合并的硬性边界，但在计算合并统计量时不要包含它们。

你的 BPE 训练函数应当返回所得到的词表和合并：

输出（Output）

- `vocab: dict[int, bytes]` 分词器词表，一个从 `int`（词表中的词元 ID）到 `bytes`（词元字节）的映射。
- `merges: list[tuple[bytes, bytes]]` 训练产生的 BPE 合并列表。每个列表项是一个字节元组 (`<token1>`, `<token2>`)，表示 `<token1>` 与 `<token2>` 被合并。这些合并应当按创建顺序排列。

要用我们提供的测试来检验你的 BPE 训练函数，你首先需要实现 `[adapters.run_train_bpe]` 处的测试适配器。然后，运行 `uv run pytest tests/test_train_bpe.py`。你的实现应当能够通过所有测试。可选地（这可能是一项较大的时间投入），你可以用某种系统级语言来实现你训练方法的关键部分，例如 C++（可考虑 `cppyy` 或 `nanobind`）或 Rust（使用 `PyO3`）。如果你这样做，要注意哪些操作需要拷贝、哪些可以直接从 Python 内存读取，并务必留下构建说明，或者确保它仅用 `pyproject.toml` 即可构建。还要注意，GPT-2 的正则表达式在大多数正则引擎中支持得并不好，并且在大多数支持它的引擎中也会太慢。我们已验证 `Oniguruma` 速度尚可且支持负向先行断言（`negative lookahead`），但 Python 的 `regex` 包如果说有什么不同的话，那就是更快。

题目 (`train_bpe_tinystories`): 在 `TinyStories` 上训练 BPE (2 分) (a) 在 `TinyStories` 数据集上训练一个字节级 BPE 分词器，使用 10,000 的最大词表大小。务必将 `TinyStories` 的 `<|endoftext|>` 特殊词元添加到词表中。将所得到的词表和合并序列化到磁盘以供进一步检查。训练花了多少时间和内存？词表中最长的词元是什么？它有意义吗？

资源要求：≤ 30 分钟（无 GPU），≤ 30 GB 内存

提示 (Hint) 在预分词期间使用 `multiprocessing`，并结合下面这两个事实，你应当能把 BPE 训练时间压到 2 分钟以内：

- (a) `<|endoftext|>` 词元在数据文件中分隔文档。
- (b) `<|endoftext|>` 词元在应用 BPE 合并之前作为特殊情形处理。

交付物：一到两句话的回答。

(b) 对你的代码进行性能剖析。分词器训练过程中的哪一部分耗时最多？

交付物：一到两句话的回答。

接下来，我们将尝试在 `OpenWebText` 数据集上训练一个字节级 BPE 分词器。和之前一样，我们建议你先查看一下数据集，以便更好地理解它的内容。

题目 (`train_bpe_expts_owt`): 在 `OpenWebText` 上训练 BPE (2 分) (a) 在 `OpenWebText` 数据集上训练一个字节级 BPE 分词器，使用 32,000 的最大词表大小。将所得到的词表和合并序列化到磁盘以供进一步检查。词表中最长的词元是什么？它有意义吗？

资源要求：≤ 12 小时（无 GPU），≤ 100 GB 内存

交付物：一到两句话的回答。

(b) 对比你在 TinyStories 上和 OpenWebText 上得到的分词器，指出它们的异同。

交付物：一到两句话的回答。

2.6 BPE 分词器：编码与解码

在本作业的上一部分中，我们实现了一个函数，在输入文本上训练一个 BPE 分词器，以获得一个分词器词表和一个 BPE 合并列表。现在，我们将实现一个 BPE 分词器，它加载所提供的词表和合并列表，并用它们在文本与词元 ID 之间进行编码和解码。

2.6.1 文本编码

用 BPE 编码文本的过程与我们训练 BPE 词表的方式相对应。其中有几个主要步骤。

第 1 步：预分词。我们首先对序列进行预分词，并将每个预词元（pre-token）表示为一个 UTF-8 字节序列，这与我们在 BPE 训练中所做的完全一样。我们将在每个预词元内部把这些字节合并为词表元素，并独立地处理每个预词元（不跨预词元边界进行合并）。

第 2 步：应用合并。接着我们取出 BPE 训练期间创建的词表元素合并序列，并按相同的创建顺序将其应用到我们的预词元上。

示例 (bpe_encoding)：BPE 编码示例 例如，假设我们的输入字符串是 'the cat ate'，我们的词表是

```
{0: b' ', 1: b'a', 2: b'c', 3: b'e', 4: b'h', 5: b't', 6: b'th', 7: b' c',  
 8: b' a', 9: b'the', 10: b' at'}
```

而我们学习到的合并是

```
[(b't', b'h'), (b' ', b'c'), (b' ', b'a'), (b'th', b'e'), (b' a', b't')].
```

首先，我们的预分词器会把这个字符串切分成 ['the', ' cat', ' ate']。然后，我们将逐个查看每个预词元并应用 BPE 合并。

第一个预词元 'the' 最初被表示为 [b't', b'h', b'e']。查看我们的合并列表，我们确定第一个可应用的合并是 (b't', b'h')，并用它把该预词元变换为 [b'th', b'e']。接着，我们回到合并列表，确定下一个可应用的合并是 (b'th', b'e')，它把该预词元变换为 [b'the']。最后，再回头查看合并列表，我们看到没有更多合并可以应用于这个字符串了（因为整个预词元已被合并成单个词元），所以我们应用 BPE 合并的工作就完成了。其对应的整数序列是 [9]。对其余预词元重复这一过程，我们看到预词元 ' cat' 在应用 BPE 合并后被表示为 [b' c', b'a', b't']，它变成整数序列 [7, 1, 5]。最后一个预词元 ' ate' 在应用 BPE 合并后是 [b' at', b'e']，它变成整数序列 [10, 3]。因此，编码我们输入字符串的最终结果是 [9, 7, 1, 5, 10, 3]。

特殊词元 (Special tokens)

你的分词器在编码文本时应当能够正确处理用户自定义的特殊词元（在构造分词器时提供）。

内存方面的考量 (Memory considerations)

假设我们想对一个无法放入内存的大型文本文件进行分词。为了高效地对这个大文件（或任何其他数据流）进行分词，我们需要把它分解成可管理的块，并依次处理每一块，使得内存复杂度是常数级的，而不是随文本大小线性增长。在这样做时，我们需要确保某个词元不会跨越块边界，否则我们得到的分词结果会与在内存中对整个序列进行分词的朴素方法不同。

2.6.2 文本解码

要将一个整数词元 ID 序列解码回原始文本，我们可以简单地在词表中查找每个 ID 对应的条目（一个字节序列），把它们拼接在一起，然后把这些字节解码为一个 Unicode 字符串。注意，输入的 ID 并不保证能映射到有效的 Unicode 字符串（因为用户可以输入任意的整数 ID 序列）。在输入的词元 ID 不能产生有效 Unicode 字符串的情况下，你应当用官方的 Unicode 替换字符 (replacement character) U+FFFD 来替换格式错误的字节。³ `bytes.decode` 的 `errors` 参数控制如何处理 Unicode 解码错误，使用 `errors='replace'` 会自动用替换标记来替换格式错误的字节。

题目 (tokenizer): 实现分词器 (15 分) **交付物:** 实现一个 `Tokenizer` 类，给定一个词表和一个合并列表，它将文本编码为整数 ID，并将整数 ID 解码为文本。你的分词器还应当支持用户提供的特殊词元（如果它们尚不在词表中，就将其追加到词表里）。我们建议采用以下接口：

`def __init__(self, vocab, merges, special_tokens=None)` 从给定的词表、合并列表以及（可选的）特殊词元列表构造一个分词器。该函数应当接受以下参数：

- `vocab: dict[int, bytes]`
- `merges: list[tuple[bytes, bytes]]`
- `special_tokens: list[str] | None = None`

`def from_files(cls, vocab_filepath, merges_filepath, special_tokens=None)` 一个类方法，它从序列化的词表和合并列表（格式与你的 BPE 训练代码的输出相同）以及（可选的）特殊词元列表构造并返回一个 `Tokenizer`。该方法应当接受以下附加参数：

- `vocab_filepath: str`
- `merges_filepath: str`
- `special_tokens: list[str] | None = None`

`def encode(self, text: str) -> list[int]` 将一段输入文本编码为一个词元 ID 序列。

³关于 Unicode 替换字符的更多信息，参见 [https://en.wikipedia.org/wiki/Specials_\(Unicode_block\)#Replacement_character](https://en.wikipedia.org/wiki/Specials_(Unicode_block)#Replacement_character)。

```
def encode_iterable(self, iterable: Iterable[str]) -> Iterator[int] 给定一个字符串的可迭代对象（例如一个 Python 文件句柄），返回一个惰性地逐个产出（yield）词元 ID 的生成器。这对于无法直接加载到内存中的大文件的内存高效分词是必需的。
```

```
def decode(self, ids: list[int]) -> str 将一个词元 ID 序列解码为文本。
```

要用我们提供的测试来检验你的 `Tokenizer`，你首先需要实现 `[adapters.get_tokenizer]` 处的测试适配器。然后，运行 `uv run pytest tests/test_tokenizer.py`。你的实现应当能够通过所有测试。

2.7 实验

题目 (`tokenizer_experiments`): 分词器实验 (4 分) (a) 从 `TinyStories` 和 `OpenWebText` 中各采样 10 个文档。使用你此前训练好的 `TinyStories` 和 `OpenWebText` 分词器（词表大小分别为 10K 和 32K），把这些采样到的文档编码为整数 ID。每个分词器的压缩比（compression ratio, 字节/词元）是多少？

交付物：一到两句话的回答。

(b) 如果你用 `TinyStories` 分词器对你的 `OpenWebText` 样本进行分词，会发生什么？比较压缩比，和/或定性地描述会发生什么。

交付物：一到两句话的回答。

(c) 估计你的分词器的吞吐量（例如以字节/秒计）。对 `Pile` 数据集（825GB 文本）进行分词要花多长时间？

交付物：一到两句话的回答。

(d) 使用你的 `TinyStories` 和 `OpenWebText` 分词器，把各自的训练集和开发集编码为整数词元 ID 序列。我们稍后将用它来训练我们的

3 Transformer 语言模型架构

语言模型 (language model) 接受一批整数词元 ID (token IDs) 序列作为输入（即形状为 `(batch_size, sequence_length)` 的 `torch.Tensor`），并返回（成批的）在词表 (vocabulary) 上归一化的概率分布（即形状为 `(batch_size, sequence_length, vocab_size)` 的 `PyTorch Tensor`），其中对每个输入词元给出的是关于下一个词的预测分布。训练语言模型时，我们用这些下一个词的预测来计算真实的下一个词与预测的下一个词之间的交叉熵损失 (cross-entropy loss)。在推理 (inference) 阶段从语言模型生成文本时，我们取最后一个时间步（即序列中的最后一项）所给出的下一个词预测分布，用它来生成序列中的下一个词元（例如取概率最高的词元、从该分布中采样等），把生成的词元追加到输入序列中，然后重复这一过程。

在本部分作业中，你将从零开始构建这个 Transformer 语言模型。我们会先对模型做一个高层次的描述，再逐步详细介绍各个组件。

3.1 Transformer LM

给定一段词元 ID 序列，Transformer 语言模型使用输入嵌入（input embedding）将词元 ID 转换为稠密向量，把嵌入后的词元送过 `num_layers` 个 Transformer 块（Transformer blocks），然后施加一个可学习的线性投影（即“输出嵌入”（output embedding）或“LM 头”（LM head）），从而产生预测的下一个词元 logits。其原理示意图 1。

（图 1）我们的 Transformer 语言模型概览：输入（*Inputs*）→ 词元嵌入（*Token Embedding*）→ `num_layers` 个 Transformer 块 → 归一化（*Norm*）→ 线性层（输出嵌入）→ *Softmax* → 输出概率（*Output Probabilities*）。

（图 2）一个预归一化（*pre-norm*）Transformer 块：输入张量形状为 $(batch_size, seq_len, d_model)$ ；先做归一化（*Norm*），经因果多头自注意力（*Causal Multi-Head Self-Attention*，带 *RoPE*）后做残差相加（*Add*）；再做归一化（*Norm*），经逐位置前馈（*Position-Wise Feed-Forward*）后再做残差相加（*Add*）；输出张量形状为 $(batch_size, seq_len, d_model)$ 。

图 1: *

图 1: Transformer 语言模型概览。 图 2: 预归一化 Transformer 块。

词元嵌入（Token Embeddings） 在最开始的一步，Transformer 把（成批的）词元 ID 序列嵌入为一串向量，这些向量包含了关于词元身份的信息（图 1 中的红色方块）。

更具体地说，给定一段词元 ID 序列，Transformer 语言模型使用一个词元嵌入层（token embedding layer）来产生一串向量。每个嵌入层接收一个形状为 $(batch_size, sequence_length)$ 的整数张量，并产生形状为 $(batch_size, sequence_length, d_model)$ 的向量序列。

预归一化 Transformer 块（Pre-norm Transformer Block） 在嵌入之后，激活值会被若干结构完全相同的神经网络层处理。一个标准的仅解码器（decoder-only）Transformer 语言模型由 `num_layers` 个完全相同的层（通常称为 Transformer “块”）构成。每个 Transformer 块接收一个形状为 $(batch_size, sequence_length, d_model)$ 的输入，并返回一个形状为 $(batch_size, sequence_length, d_model)$ 的输出。每个块通过自注意力（self-attention）在序列上聚合信息，并通过前馈层（feed-forward layers）对其做非线性变换。

在经过 `num_layers` 个 Transformer 块之后，我们会取最终的激活值并把它们转化为词表上的一个分布。

我们将实现“预归一化”（pre-norm）Transformer 块（详见 §3.4），它还要求在最后一个 Transformer 块之后使用层归一化（layer normalization，详见下文），以确保其输出被恰当地缩放。

在这次归一化之后，我们会用一个标准的可学习线性变换把 Transformer 块的输出转换为预测的下一个词元 logits（参见，例如，A. Radford 等人著作中的式 (2)）。

3.2 说明：批处理、Einsum 与高效计算

在整个 Transformer 中，我们会对许多“类批”（batch-like）输入反复执行同一种计算。下面是几个例子：

- **一个批次的各元素**：我们对每个批次元素施加同一个 Transformer 前向操作。
- **序列长度**：像 RMSNorm 和前馈这样的“逐位置”（position-wise）操作，在序列的每个位置上以完全相同的方式运算。
- **注意力头（attention heads）**：注意力操作在多个注意力头上以“多头”（multi-headed）注意力的方式被批处理。

有一种符合人体工学的方式来执行这类操作是很有用的，它既能充分利用 GPU，又易于阅读和理解。许多 PyTorch 操作可以在张量起始处接受多余的“类批”维度，并高效地在这些维度上重复或广播（broadcast）该操作。

举例来说，假设我们在做一个逐位置的、成批的操作。我们有一个形状为 $(\text{batch_size}, \text{sequence_length}, \text{d_model})$ 的“数据张量” D ，并希望对它与一个形状为 $(\text{d_model}, \text{d_model})$ 的矩阵 A 做成批的向量-矩阵乘法。在这种情形下， $D @ A$ 会执行一次成批的矩阵乘法，这在 PyTorch 中是一种高效的基本运算，其中 $(\text{batch_size}, \text{sequence_length})$ 这两个维度被批处理掉了。

正因如此，假定你的函数可能会被传入额外的类批维度，并把这些维度保持在 PyTorch 形状的起始处，是很有帮助的。为了把张量组织成可以这样批处理的形式，可能需要用许多步 `view`、`reshape` 和 `transpose` 来重塑它们。这会有点麻烦，而且往往很难看清代码在做什么、以及张量的形状是什么。

一个更符合人体工学的选择是在 `torch.einsum` 中使用 `einsum` 记号，或者干脆使用与框架无关的库，如 `einops` 或 `einx`。其中两个关键操作是 `einsum`（可对任意维度的输入张量做张量收缩（tensor contraction））和 `rearrange`（可对任意维度进行重排、拼接和拆分）。事实证明，机器学习中几乎所有操作都是维度调配与张量收缩的某种组合，外加偶尔出现的（通常是逐点的）非线性函数。这意味着，使用 `einsum` 记号时，你的许多代码都能变得更可读、更灵活。

我们强烈建议你在本课程中学习并使用 `einsum` 记号。此前没接触过 `einsum` 记号的同学应该使用 `einops`（文档见其官网），已经熟悉 `einops` 的同学则应学习更通用的 `einx`。⁴ 这两个包都已预装在我们提供的环境中。

下面给出一些如何使用 `einsum` 记号的示例。它们是对 `einops` 文档的补充，你应当先阅读该文档。

示例 (`einstein_example1`)：用 `einops.einsum` 做成批矩阵乘法

```
import torch
from einops import rearrange, einsum

## Basic implementation
Y = D @ A.T
# Hard to tell the input and output shapes and what they mean.
# What shapes can D and A have, and do any of these have unexpected behavior?

## Einsum is self-documenting and robust
#           D           A           ->           Y
```

⁴值得一提的是，`einops` 有大量支持，而 `einx` 没有那么久经考验。如果你在使用 `einx` 时发现任何局限或缺陷，可以随时退回到使用 `einops` 加一些更朴素的 PyTorch。

```
Y = einsum(D, A, "batch sequence d_in, d_out d_in -> batch sequence d_out")

## Or, a batched version where D can have any leading dimensions but A is constrained
.
Y = einsum(D, A, "... d_in, d_out d_in -> ... d_out")
```

示例 (einstein_example2): 用 `einops.rearrange` 做广播运算 我们有一批图像, 对每张图像都想根据某个缩放因子生成 10 个变暗的版本:

```
images = torch.randn(64, 128, 128, 3) # (batch, height, width, channel)
dim_by = torch.linspace(start=0.0, end=1.0, steps=10)

## Reshape and multiply
dim_value = rearrange(dim_by, "dim_value -> 1 dim_value 1 1 1")
images_rearr = rearrange(images, "b height width channel -> b 1 height width channel"
)
dimmed_images = images_rearr * dim_value

## Or in one go:
dimmed_images = einsum(
    images, dim_by,
    "batch height width channel, dim_value -> batch dim_value height width channel"
)
```

示例 (einstein_example3): 用 `einops.rearrange` 做像素混合 假设我们有一批以形状 (batch, height, width, channel) 张量表示的图像, 并希望对图像的所有像素做一次线性变换, 但这一变换应对每个通道独立地进行。我们的线性变换用一个形状为 (height * width, height * width) 的矩阵 B 表示。

```
channels_last = torch.randn(64, 32, 32, 3) # (batch, height, width, channel)
B = torch.randn(32*32, 32*32)

## Rearrange an image tensor for mixing across all pixels
channels_last_flat = channels_last.view(
    -1, channels_last.size(1) * channels_last.size(2), channels_last.size(3)
)
channels_first_flat = channels_last_flat.transpose(1, 2)
channels_first_flat_transformed = channels_first_flat @ B.T
channels_last_flat_transformed = channels_first_flat_transformed.transpose(1, 2)
channels_last_transformed = channels_last_flat_transformed.view(*channels_last.shape)
```

改用 `einops`:

```
height = width = 32
```

```

## Rearrange replaces clunky torch view + transpose
channels_first = rearrange(
    channels_last,
    "batch height width channel -> batch channel (height width)"
)
channels_first_transformed = einsum(
    channels_first, B,
    "batch channel pixel_in, pixel_out pixel_in -> batch channel pixel_out"
)
channels_last_transformed = rearrange(
    channels_first_transformed,
    "batch channel (height width) -> batch height width channel",
    height=height, width=width
)

```

或者, 如果你觉得够大胆: 用 `einx.dot` (`einx` 中对应 `einops.einsum` 的操作) 一步到位:

```

height = width = 32
channels_last_transformed = einx.dot(
    "batch row_in col_in channel, (row_out col_out) (row_in col_in)"
    "-> batch row_out col_out channel",
    channels_last, B,
    col_in=width, col_out=width
)

```

这里的第一种实现可以通过在前后加注释来注明输入和输出形状以作改进, 但这样做笨拙且容易出错。而使用 `einsum` 记号时, 文档即实现!

`einsum` 记号能处理任意的输入批处理维度, 但还有一个关键好处: 它具有自文档(self-documenting)特性。在使用 `einsum` 记号的代码中, 输入和输出张量的相关形状要清晰得多。对于其余的张量, 你可以考虑使用张量类型提示 (type hints), 例如使用 `jaxtyping` 库 (它并不专属于 JAX)。

我们将在作业二中更多地讨论使用 `einsum` 记号在性能上的影响, 但现在你只需知道: 它们几乎总是优于其他替代方案!

3.2.1 数学记号与内存排序

许多机器学习论文在记号中使用行向量 (row vectors), 这样得到的表示与 NumPy 和 PyTorch 默认采用的行主序 (row-major) 内存排序契合得很好。在使用行向量时, 一个线性变换形如

$$y = xW^{\top}, \quad (1)$$

其中行主序矩阵 $W \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}$, 行向量 $x \in \mathbb{R}^{1 \times d_{\text{in}}}$ 。注意, 这让我们可以通过增大 x 的最外层维度来批处理输入, 也就是说, 可以把向量输入 x 替换为矩阵输入 $X \in \mathbb{R}^{\text{batch} \times d_{\text{in}}}$ 。

在线性代数中, 一般更常用列向量 (column vectors), 此时线性变换形如

$$y = Wx, \quad (2)$$

其中行主序矩阵 $W \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}$ ，列向量 $x \in \mathbb{R}^{d_{\text{in}}}$ 。要在这种约定下批处理输入， x 的批维度必须放在最后，于是 x 需要被替换为矩阵 $\tilde{X} \in \mathbb{R}^{d_{\text{in}} \times \text{batch}}$ 。

在本次作业的数学记号中，我们将主要使用列向量，因为数学一般遵循这一约定。你应当记住：如果你想使用普通的矩阵乘法记号，就必须像式 (1) 中行向量约定那样对矩阵施加一个转置，因为 PyTorch 使用行主序内存排序。如果你用 einsum 来做线性代数运算，那么只要你正确地标注了各个轴，这就不成问题。顺带一提，值得注意的是，其他语言/线性代数包，如 Matlab、Julia 和 Fortran，都使用列主序（column-major）内存排序，意味着批处理维度排在最后；而 Python 及相关的包采用了 C 标准的行主序排序。

3.3 基础模块：Linear 与 Embedding 模块

3.3.1 参数初始化

要有效地训练神经网络，往往需要对模型参数做精心的初始化——糟糕的初始化会导致诸如梯度消失（vanishing gradients）或梯度爆炸（exploding gradients）之类的不良行为。预归一化（pre-norm）Transformer 对初始化异乎寻常地稳健，但初始化仍可能对训练速度和收敛产生显著影响。由于这次作业已经很长，我们把细节留到作业三，这里只给你一些在大多数情形下都应表现良好的近似初始化方法。目前，请使用：

- **Linear 权重：** $\mathcal{N}\left(\mu = 0, \sigma^2 = \frac{2}{d_{\text{in}} + d_{\text{out}}}\right)$ ，在 $[-3\sigma, 3\sigma]$ 处截断。
- **Embedding：** $\mathcal{N}(\mu = 0, \sigma^2 = 1)$ ，在 $[-3, 3]$ 处截断。
- **RMSNorm：** 1（全 1 初始化）。

你应当使用 `torch.nn.init.trunc_normal_` 来初始化截断正态（truncated normal）权重。

3.3.2 Linear 模块

线性层（Linear layers）是 Transformer 乃至一般神经网络的基础构件。首先，你将实现自己的 Linear 类，它继承自 `torch.nn.Module` 并执行一个线性变换：

3.3.3 嵌入模块（Embedding Module）

如上所述，Transformer 的第一层是一个嵌入层（embedding layer），它将整数词元 ID（token ID）映射到维度为 `d_model` 的向量空间。我们将实现一个继承自 `torch.nn.Module` 的自定义 Embedding 类（因此你不应使用 `nn.Embedding`）。`forward` 方法应当通过对一个形状为（`vocab_size`, `d_model`）的嵌入矩阵进行索引来为每个词元 ID 选取对应的嵌入向量；索引时使用一个形状为（`batch_size`, `sequence_length`）的 `torch.LongTensor` 词元 ID 张量。

题目 (embedding)：实现嵌入模块（1 分）**交付物：**实现继承自 `torch.nn.Module` 并执行嵌入查找的 Embedding 类。你的实现应遵循 PyTorch 内置 `nn.Embedding` 模块的接口。我们推荐如下接口：

```
def __init__(self, num_embeddings, embedding_dim, device=None, dtype=None)
```

构造一个嵌入模块。该函数应接受以下参数：

- `num_embeddings`: `int` 词表大小
- `embedding_dim`: `int` 嵌入向量的维度，即 d_{model}
- `device`: `torch.device` | `None` = `None` 存放参数的设备
- `dtype`: `torch.dtype` | `None` = `None` 参数的数据类型

```
def forward(self, token_ids: torch.Tensor) -> torch.Tensor
```

 为给定的词元 ID 查找嵌入向量。

请务必做到：

- 继承 `nn.Module`；
- 调用父类构造函数；
- 将你的嵌入矩阵初始化为一个 `nn.Parameter`；
- 存储嵌入矩阵时令 `d_model` 作为最后一维；
- 当然，不要使用 `nn.Embedding` 或 `nn.functional.embedding`。

同样，初始化时使用上文给出的设置，并使用 `torch.nn.init.trunc_normal_` 来初始化权重。为测试你的实现，请实现位于 `[adapters.run_embedding]` 的测试适配器（test adapter）。然后运行 `uv run pytest -k test_embedding`。

3.4 预归一化 Transformer 块（Pre-Norm Transformer Block）

每个 Transformer 块都有两个子层（sub-layer）：一个多头自注意力（multi-head self-attention）机制和一个位置前馈网络（position-wise feed-forward network）（见 A. Vaswani 等人，2017，第 3.1 节）。

在最初的 Transformer 论文中，模型在两个子层各自周围使用一个残差连接（residual connection），其后接一个层归一化（layer normalization）。这种架构通常被称为“后归一化”（post-norm）Transformer，因为层归一化作用于子层的输出。然而，大量工作发现，将层归一化从每个子层的输出处移到每个子层的输入处（并在最后一个 Transformer 块之后额外加一个层归一化）能够改善 Transformer 的训练稳定性（T. Q. Nguyen 等人，2019；R. Xiong 等人，2020）——这种“预归一化”（pre-norm）Transformer 块的可视化表示见图 2。随后，每个 Transformer 块子层的输出通过残差连接被加回到该子层的输入上（见 A. Vaswani 等人，第 5.4 节）。对预归一化的一个直观理解是：存在一条从输入嵌入一直通向 Transformer 最终输出的、不含任何归一化的干净“残差流”（residual stream），据称这能改善梯度流动。如今，这种预归一化 Transformer 已成为当代语言模型（例如 GPT-3、LLaMA、PaLM 等）的标准做法，因此我们将实现这一变体。我们将逐一介绍预归一化 Transformer 块的各个组件，并依次实现它们。

3.4.1 均方根层归一化 (Root Mean Square Layer Normalization)

A. Vaswani 等人最初的 Transformer 实现使用层归一化 (J. L. Ba 等人, 2016) 来归一化激活值。遵循 H. Touvron 等人 (2023) 的做法, 我们将使用均方根层归一化 (root mean square layer normalization, RMSNorm; B. Zhang 等人, 公式 4) 来进行层归一化。给定一个激活向量 $a \in \mathbb{R}^{d_{\text{model}}}$, RMSNorm 将按如下方式重新缩放每个激活值 a_i :

$$\text{RMSNorm}(a_i) = \frac{a_i}{\text{RMS}(a)} g_i, \quad (4)$$

其中 $\text{RMS}(a) = \sqrt{\frac{1}{d_{\text{model}}} \sum_{i=1}^{d_{\text{model}}} a_i^2 + \epsilon}$ 。这里, g_i 是一个可学习的“增益” (gain) 参数 (这样的参数共有 d_{model} 个), 而 ϵ 是一个超参数, 通常固定为 $1e-5$ 。

为防止对输入求平方时发生溢出, 你应将输入向上转型 (upcast) 为 `torch.float32`。总体而言, 你的 `forward` 方法应当形如:

```
in_dtype = x.dtype
x = x.to(torch.float32)

# Your code here performing RMSNorm
...
result = ...

# Return the result in the original dtype
return result.to(in_dtype)
```

题目 (rmsnorm): 均方根层归一化 (1 分) **交付物:** 将 RMSNorm 实现为一个 `torch.nn.Module`。我们推荐如下接口:

`def __init__(self, d_model: int, eps: float = 1e-5, device=None, dtype=None)`
构造 RMSNorm 模块。该函数应接受以下参数:

- `d_model: int` 模型的隐藏维度
- `eps: float = 1e-5` 用于数值稳定性的 epsilon 值
- `device: torch.device | None = None` 存放参数的设备
- `dtype: torch.dtype | None = None` 参数的数据类型

`def forward(self, x: torch.Tensor) -> torch.Tensor` 处理一个形状为 $(\text{batch_size}, \text{sequence_length}, d_{\text{model}})$ 的输入张量, 并返回一个形状相同的张量。

注意: 如上所述, 请记得在执行归一化之前将输入向上转型为 `torch.float32` (并在之后向下转型回原始 `dtype`)。

为测试你的实现, 请实现位于 `[adapters.run_rmsnorm]` 的测试适配器。然后运行 `uv run pytest -k test_rmsnorm`。

3.4.2 位置前馈网络（Position-Wise Feed-Forward Network）

图 3：比较 SiLU（又名 Swish）与 ReLU 激活函数。

（图示对比三条曲线：SiLU : $f(x) = x \sigma(x)$ ；恒等映射 Identity : $f(x) = x$ ；
ReLU : $f(x) = \max(0, x)$ ）

在最初的 Transformer 论文中（A. Vaswani 等人第 3.3 节），Transformer 的前馈网络由两个线性变换组成，二者之间夹有一个 ReLU 激活（ $\text{ReLU}(x) = \max(0, x)$ ）。在那一最初架构中，前馈层内部维度通常是输入维度的 4 倍。

然而，与这一最初设计相比，现代语言模型往往引入两项主要改动：使用另一种激活函数，并采用一种门控（gating）机制。具体而言，我们将实现 Llama 3（A. Grattafiori 等人，2024）和 Qwen 2.5（A. Yang 等人，2024）等大语言模型所采用的“SwiGLU”激活函数，它将 SiLU（常被称为 Swish）激活与一种称为门控线性单元（Gated Linear Unit, GLU）的门控机制结合起来。我们还将省略线性层中有时使用的偏置项，这遵循自 PaLM（A. Chowdhery 等人，2022）和 LLaMA（H. Touvron 等人，2023）以来大多数现代大语言模型的做法。

SiLU 或 Swish 激活函数（D. Hendrycks 等人，2016；S. Elfwing 等人，2017）定义如下：

$$\text{SiLU}(x) = x \cdot \sigma(x) = \frac{x}{1 + e^{-x}} \quad (5)$$

如图 3 所示，SiLU 激活函数与 ReLU 激活函数类似，但在零点处是光滑的。

门控线性单元（GLU）最初由 Y. N. Dauphin 等人定义为：一个经过 sigmoid 函数的线性变换与另一个线性变换的逐元素乘积：

$$\text{GLU}(x, W_1, W_2) = \sigma(W_1 x) \odot W_2 x, \quad (6)$$

其中 $\odot$ 表示逐元素乘法。门控线性单元被认为能够“通过为梯度提供一条线性通路、同时保留非线性能力，来缓解深层架构中的梯度消失问题”。

3.4.3 相对位置编码（Relative Positional Embeddings）

为了把位置信息注入模型，我们将实现旋转位置编码（Rotary Position Embeddings）[J. Su 等人，2021]，通常称为 RoPE。对于位于词元位置 i 处的某个查询词元 $q^{(i)} = W_q x^{(i)} \in \mathbb{R}^d$ ，我们将对其施加一个成对旋转矩阵（pairwise rotation matrix） R_i ，得到 $q'^{(i)} = R_i q^{(i)} = R_i W_q x^{(i)}$ 。这里， R_i 会把成对的嵌入元素 $q_{2k-1:2k}^{(i)}$ 当作二维向量，按角度 $\theta_{i,k} = \frac{i}{\Theta(2k-2)/d}$ 旋转，其中 $k \in \{1, \dots, d/2\}$ ， Θ 为某个常数。因此，我们可以把 R^i 看作一个大小为 $d \times d$ 的分块对角矩阵（block-diagonal matrix），其各个分块为 R_k^i （ $k \in \{1, \dots, \frac{d}{2}\}$ ），其中

$$R_k^i = \begin{pmatrix} \cos(\theta_{i,k}) & -\sin(\theta_{i,k}) \\ \sin(\theta_{i,k}) & \cos(\theta_{i,k}) \end{pmatrix} \quad (8)$$

于是我们得到完整的旋转矩阵

$$R^i = \begin{pmatrix} R_1^i & 0 & 0 & \cdots & 0 \\ 0 & R_2^i & 0 & \cdots & 0 \\ 0 & 0 & R_3^i & \cdots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & 0 & \cdots & R_{d/2}^i \end{pmatrix}, \quad (9)$$

其中各个 0 代表 2×2 的零矩阵。虽然我们可以直接构造完整的 $d \times d$ 矩阵，但一个好的实现应当利用该矩阵的性质，以更高效的方式实现这一变换。由于我们只关心给定序列内各词元之间的相对旋转，因此可以在不同层、不同批次之间复用我们为 $\cos(\theta_{i,k})$ 与 $\sin(\theta_{i,k})$ 计算出的数值。如果你想对其进行优化，可以使用一个被所有层共同引用的单一 RoPE 模块，并让它在初始化时通过 `self.register_buffer(persistent=False)` 创建一个预先计算好的二维 sin 与 cos 数值缓冲区 (buffer)，而不是使用 `nn.Parameter` (因为我们并不想学习这些固定的余弦与正弦值)。我们对 $q^{(i)}$ 所做的旋转过程，会以完全相同的方式对 $k^{(j)}$ 进行，按照对应的 R_j 进行旋转。注意，这一层没有任何可学习参数。

题目 (rope): 实现 RoPE (2 分) **交付物:** 实现一个类 `RotaryPositionalEmbedding`, 将 RoPE 施加到输入张量上。

推荐采用如下接口:

`def __init__(self, theta: float, d_k: int, max_seq_len: int, device=None)` 构造 RoPE 模块, 并在需要时创建缓冲区。

- `theta: float` RoPE 所用的 Θ 值
- `d_k: int` 查询向量与键向量的维度
- `max_seq_len: int` 将要输入的最大序列长度
- `device: torch.device | None = None` 用于存储缓冲区的设备

`def forward(self, x: torch.Tensor, token_positions: torch.Tensor) -> torch.Tensor` 处理一个形状为 $(\dots, \text{seq_len}, d_k)$ 的输入张量, 并返回一个形状相同的张量。注意, 你应当能容忍 x 带有任意数量的批次维度 (batch dimension)。你可以假设 `token positions` 是一个形状为 $(\dots, \text{seq_len})$ 的张量, 指定了 x 沿序列维度的各词元位置。你应当使用 `token positions` 来沿序列维度对 (可能已预先计算好的) cos 与 sin 张量进行切片。为测试你的实现, 请补全 `[adapters.run_rope]`, 并确保通过 `uv run pytest -k test_rope`。

3.4.4 缩放点积注意力 (Scaled Dot-Product Attention)

我们现在将实现 A. Vaswani 等人 [8] (第 3.2.1 节) 所描述的缩放点积注意力 (scaled dot-product attention)。作为一个预备步骤, 注意力 (attention) 运算的定义将用到 softmax, 这是一种把一组未归一化的分数向量转化为归一化分布的运算:

$$\text{softmax}(v)_i = \frac{\exp(v_i)}{\sum_{j=1}^n \exp(v_j)}. \quad (10)$$

注意, 对于较大的数值, $\exp(v_i)$ 可能会变成 `inf` (此时 $\frac{\text{inf}}{\text{inf}} = \text{NaN}$)。我们可以利用一个事实来规避这一点: softmax 运算对于给所有输入加上任意常数 c 是不变的。我们可以利用这一性质来获得数值稳定性——通常我们会从 v 的所有元素中减去 v 的最大项, 使新的最大项变为 0。下面你将实现 softmax, 并使用这个技巧来保证数值稳定性。

题目 (softmax): 实现 softmax (1 分) **交付物:** 编写一个函数, 对张量施加 softmax 运算。你的函数应当接收两个参数: 一个张量和一个维度 i , 并对输入张量的第 i 维施加 softmax。输出张量的形状应与输入张量相同, 但其第 i 维现在将是一个归一化的概率分布。请使用从第 i 维所有元素中减去该维最大值的技巧, 以避免数值稳定性问题。

为测试你的实现, 请补全 `[adapters.run_softmax]`, 并确保通过 `uv run pytest -k test_softmax_matches_pytorch`。

现在我们可以从数学上把注意力运算定义如下:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V \quad (11)$$

其中 $Q \in \mathbb{R}^{n \times d_k}$, $K \in \mathbb{R}^{m \times d_k}$, $V \in \mathbb{R}^{m \times d_v}$ 。这里, Q 、 K 、 V 都是该运算的输入——注意它们并不是可学习参数。

掩码 (Masking): 有时对注意力运算的输出进行掩码处理会很方便。一个掩码 (mask) 应当具有形状 $M \in \{\text{True}, \text{False}\}^{n \times m}$, 这个布尔矩阵的每一行 i 都指明了查询 i 应当关注哪些键。按照惯例 (且略微令人困惑), 位置 (i, j) 处取值为 True 表示查询 i 会关注键 j , 取值为 False 表示查询不关注该键。换言之, “信息流动” 发生在取值为 True 的 (i, j) 对上。例如, 考虑一个 1×3 的掩码矩阵, 其元素为 `[[True, True, False]]`。这唯一的查询向量只会关注前两个键。

在计算上, 使用掩码会比在子序列上计算注意力高效得多; 我们可以这样做: 取 softmax 之前的值 $(QK^T / \sqrt{d_k})$, 并对掩码矩阵中任何取值为 False 的项加上 $-\infty$ 。

题目 (scaled_dot_product_attention): 实现缩放点积注意力 (5 分) **交付物:** 实现缩放点积注意力函数。你的实现应当能处理形状为 `(batch_size, ..., seq_len, d_k)` 的键与查询, 以及形状为 `(batch_size, ..., seq_len, d_v)` 的值, 其中 `...` 代表任意数量的其他类批次 (batch-like) 维度 (如有提供)。该实现应返回一个形状为 `(batch_size, ..., seq_len, d_v)` 的输出。关于类批次维度的讨论, 见 §3.2。

你的实现还应支持一个由用户提供的、可选的布尔掩码, 其形状为 `(seq_len, seq_len)`。掩码取值为 True 的各位置上的注意力概率应共同求和为 1, 而掩码取值为 False 的各位置上的注意力概率应为零。

为针对我们提供的测试来测试你的实现, 你需要补全测试适配器 `[adapters.run_scaled_dot_product_attention]`。`uv run pytest -k test_scaled_dot_product_attention` 会在三阶输入张量上测试你的实现, 而 `uv run pytest -k test_4d_scaled_dot_product_attention` 会在四阶输入张量上测试你的实现。

3.4.5 因果多头自注意力 (Causal Multi-Head Self-Attention)

我们将实现 A. Vaswani 等人 [8] 第 3.2.2 节所描述的多头自注意力 (multi-head self-attention)。回顾一下, 多头注意力运算在数学上定义如下:

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h) \quad (12)$$

$$\text{for } \text{head}_i = \text{Attention}(Q_i, K_i, V_i) \quad (13)$$

其中 Q_i, K_i, V_i 分别是 Q, K, V 的嵌入维度上的第 i 个切片 ($i \in \{1, \dots, h\}$)，大小为 d_k 或 d_v 。这里的 Attention 即第 3.4.4 节所定义的缩放点积注意力运算。由此我们可以构造出多头自注意力运算：

$$\text{MultiHeadSelfAttention}(x) = W_O \text{MultiHead}(W_Q x, W_K x, W_V x) \quad (14)$$

这里，可学习参数为 $W_Q \in \mathbb{R}^{hd_k \times d_{\text{model}}}$ 、 $W_K \in \mathbb{R}^{hd_k \times d_{\text{model}}}$ 、 $W_V \in \mathbb{R}^{hd_v \times d_{\text{model}}}$ ，以及 $W_O \in \mathbb{R}^{d_{\text{model}} \times hd_v}$ 。由于在多头注意力运算中 Q, K, V 是被切片的，我们可以把 W_Q, W_K, W_V 看作沿输出维度为每个头分离开来。当这部分能正常工作时，你应当只用总共三次矩阵乘法就计算出键、值、查询的投影。⁵

因果掩码 (Causal masking) 你的实现应当阻止模型关注序列中的未来词元。换言之，如果给模型一个词元序列 $t_1, \dots, t_n$ ，而我们想为前缀 $t_1, \dots, t_i$ (其中 $i < n$) 计算下一个词的预测，那么模型不应能够访问（关注）位置 $t_{i+1}, \dots, t_n$ 处的词元表示，因为在推理 (inference) 阶段生成文本时模型并无法访问这些词元（而且这些未来词元会泄露真实下一个词的身份信息，使语言建模预训练目标变得平凡）。对于一个输入词元序列 $t_1, \dots, t_n$ ，我们可以通过把多头自注意力运行 n 次（对应序列中 n 个不同的前缀）来天真地阻止对未来词元的访问。但我们将改用因果注意力掩码 (causal attention masking)，它允许词元 i 关注序列中所有满足 $j \leq i$ 的位置 j 。你可以使用 `torch.triu` 或广播式的索引比较来构造该掩码，并且你应当利用一个事实：你在第 3.4.4 节中实现的缩放点积注意力已经支持注意力掩码。

施加 RoPE (Applying RoPE) RoPE 应当被施加到查询向量和键向量上，但不施加到值向量上。此外，头 (head) 维度应当作为一个批次维度来处理，因为在多头注意力中，注意力是为每个头独立施加的。这意味着对每个头的查询向量和键向量都应当施加完全相同的 RoPE 旋转。

题目 (multihead_self_attention)：实现因果多头自注意力 (5 分) **交付物**：把因果多头自注意力实现为一个 `torch.nn.Module`。你的实现应当（至少）接收如下参数：

- `d_model`: `int` Transformer 块输入的维度。
- `num_heads`: `int` 多头自注意力中使用的头数。

参照 A. Vaswani 等人 [8]，设 $d_k = d_v = \frac{d_{\text{model}}}{h}$ 。为针对我们提供的测试来测试你的实现，请实现测试适配器 `[adapters.run_multihead_self_attention]`。然后运行 `uv run pytest -k test_multihead_self_attention` 来测试你的实现。

3.5 完整的 Transformer 语言模型 (The Full Transformer LM)

我们先从组装 Transformer 块 (Transformer block) 开始 (回顾图 2 会有所帮助)。一个 Transformer 块包含两个“子层” (sub-layer)，一个用于多头自注意力，另一个用于 SwiGLU 前馈网

⁵作为一个进阶目标 (stretch goal)，可尝试把键、查询、值的投影合并到单个权重矩阵中，这样你就只需要一次矩阵乘法。

络 (feed-forward network)。在每个子层中, 我们首先执行 RMSNorm, 然后执行主运算 (MHA/FF), 最后加上残差连接 (residual connection)。

具体来说, Transformer 块的前半部分 (第一个”子层”) 应当实现如下一组更新, 从输入 x 产生输出 y :

$$y = x + \text{MultiHeadSelfAttention}(\text{RMSNorm}(x)). \quad (15)$$

题目 (transformer_block): 实现 Transformer 块 (3 分) 实现第 3.4 节所描述、并在图 2 中所示的预归一化 (pre-norm) Transformer 块。你的 Transformer 块应当 (至少) 接收如下参数:

- `d_model`: `int` Transformer 块输入的维度。
- `num_heads`: `int` 多头自注意力中使用的头数。
- `d_ff`: `int` 逐位置前馈层内部层的维度。

为测试你的实现, 请实现适配器 `[adapters.run_transformer_block]`。然后运行 `uv run pytest -k test_transformer_block` 来测试你的实现。

交付物: 能通过所提供测试的 Transformer 块代码。

现在我们把这些块拼装起来, 遵循图 1 中的高层示意图。按照第 3.1.0.1 节中我们对嵌入 (embedding) 的描述, 将其输入到 `num_layers` 个 Transformer 块中, 然后再传入最后的层归一化 (layer norm) 和语言模型头 (LM head), 以得到一个在词表上的未归一化分布 (即 logits)。

题目 (transformer_lm): 实现 Transformer 语言模型 (3 分) 是时候把所有部分整合到一起了! 实现第 3.1 节所描述、并在图 1 中所示的 Transformer 语言模型。你的实现至少应当接收前面提到的 Transformer 块的所有构造参数, 以及如下这些额外参数:

- `vocab_size`: `int` 词表大小, 用于确定词元嵌入矩阵的维度。
- `context_length`: `int` 最大上下文长度, 用于确定 RoPE 的 `sin` 与 `cos` 缓冲区的维度。
- `num_layers`: `int` 要使用的 Transformer 块的数量。

为针对我们提供的测试来测试你的实现, 你需要先实现测试适配器 `[adapters.run_transformer_lm]`。然后运行 `uv run pytest -k test_transformer_lm` 来测试你的实现。

交付物: 一个能通过上述测试的 Transformer 语言模型模块。

资源核算 (Resource accounting) 能够理解 Transformer 的各个部分如何消耗计算和内存是很有用的。我们将一步步进行一些基本的”FLOPs 核算”。Transformer 中绝大多数的 FLOPs 都是矩阵乘法, 因此我们的核心方法很简单:

1. 写下 Transformer 前向传播中的所有矩阵乘法。

2. 把每个矩阵乘法换算为所需的 FLOPs。

对于第二步，下面这个事实会很有用：

规则 给定 $A \in \mathbb{R}^{m \times n}$ 与 $B \in \mathbb{R}^{n \times p}$ ，矩阵-矩阵乘积 AB 需要 $2mnp$ 次 FLOPs。

为说明这一点，注意到 $(AB)[i, j] = A[i, :] \cdot B[:, j]$ ，而这个点积需要 n 次加法和 n 次乘法（共 $2n$ 次 FLOPs）。然后，由于矩阵-矩阵乘积 AB 有 $m \times p$ 个元素，FLOPs 的总数为 $(2n)(mp) = 2mnp$ 。

现在，在做下一道题之前，逐一过一遍你的 Transformer 块和 Transformer 语言模型的每个组成部分，列出所有矩阵乘法及其相应的 FLOPs 开销，会很有帮助。

题目 (transformer_accounting): Transformer 语言模型资源核算 (5 分) (a) 考虑一个使用我们作业架构、规模与 GPT-2 XL 相当的模型，其配置如下：

vocab_size:	50,257
context_length:	1,024
num_layers:	48
d_model:	1,600
num_heads:	25
d_ff:	4,288 (最接近 $\frac{8}{3} \times 1,600$ 的 64 的倍数)

假设我们用这个配置构造模型。我们的模型会有多少个可训练参数？假设每个参数用单精度浮点数 (single-precision floating point) 表示，仅仅加载这个模型需要多少内存？

交付物：一到两句话的回答。

(b) 找出完成我们这个 GPT-2 XL 形状模型的一次前向传播所需的矩阵乘法。这些矩阵乘法总共需要多少 FLOPs？假设我们的输入序列含有 `context_length` 个词元。

4 训练 Transformer 语言模型

我们现在已经具备了对数据进行预处理（通过分词器）和构建模型（Transformer）的步骤。接下来要做的，是搭建支撑训练的全部代码。这包括以下几个部分：

- **损失 (Loss)：**我们需要定义损失函数（交叉熵，cross-entropy）。
- **优化器 (Optimizer)：**我们需要定义用于最小化该损失的优化器（AdamW）。
- **训练循环 (Training loop)：**我们需要全部配套的基础设施，用于加载数据、保存检查点 (checkpoint) 以及管理训练过程。

4.1 交叉熵损失

回顾一下，Transformer 语言模型为每个长度为 $m+1$ 的序列 x 和 $i = 1, \dots, m$ 定义了一个分布 $p_{\theta}(x_{i+1} | x_{1:i})$ 。给定一个由长度为 $m+1$ 的序列组成的训练集 D ，我们定义标准的交叉熵（负

对数似然, negative log-likelihood) 损失函数:

$$\ell(\theta; D) = \frac{1}{|D|} \sum_{x \in D} \sum_{i=1}^m -\log p_{\theta}(x_{i+1} | x_{1:i}). \quad (16)$$

(注意, Transformer 的一次前向传播会同时得到所有 $i = 1, \dots, m$ 对应的 $p_{\theta}(x_{i+1} | x_{1:i})$)
 具体而言, Transformer 为每个位置 i 计算 logits $o_i \in \mathbb{R}^{\text{vocab_size}}$, 由此得到:⁶

$$p(x_{i+1} | x_{1:i}) = \text{softmax}(o_i)[x_{i+1}] = \frac{\exp(o_i[x_{i+1}])}{\sum_{a=1}^{\text{vocab_size}} \exp(o_i[a])}. \quad (17)$$

交叉熵损失一般是相对于 logits 向量 $o_i \in \mathbb{R}^{\text{vocab_size}}$ 与目标 x_{i+1} 来定义的。⁷

实现交叉熵损失需要像 softmax 那样小心处理数值问题。

题目 (cross_entropy): 实现交叉熵 (1 分) **交付物:** 编写一个计算交叉熵损失的函数, 它接收预测的 logits (o_i) 和目标 (x_{i+1}), 并计算交叉熵 $\ell_i = -\log \text{softmax}(o_i)[x_{i+1}]$ 。你的函数应处理以下情况:

- 为了数值稳定性, 减去最大的元素。
- 尽可能抵消 log 与 exp。
- 处理任何额外的批次 (batch) 维度, 并返回该批次上的平均值。与第 3.2 节一样, 我们假设批次类维度总是排在最前面, 位于词表大小维度之前。

实现 [adapters.run_cross_entropy], 然后运行 `uv run pytest -k test_cross_entropy` 来测试你的实现。

困惑度 (Perplexity)

交叉熵足以用于训练, 但当我们评估模型时, 还希望报告困惑度 (perplexity)。对于一个长度为 m 、各位置承受交叉熵损失 $\ell_1, \dots, \ell_m$ 的序列:

$$\text{perplexity} = \exp\left(\frac{1}{m} \sum_{i=1}^m \ell_i\right). \quad (18)$$

4.2 SGD 优化器

现在我们有损失函数, 接下来开始探索优化器。最简单的基于梯度的优化器是随机梯度下降 (Stochastic Gradient Descent, SGD)。我们从随机初始化的参数 θ_0 开始。然后对每一步 $t = 0, \dots, T-1$, 执行以下更新:

$$\theta_{t+1} \leftarrow \theta_t - \alpha_t \nabla L(\theta_t; B_t), \quad (19)$$

其中 B_t 是从数据集 D 中采样的一个随机数据批次, 学习率 α_t 与批次大小 $|B_t|$ 是超参数。

⁶注意 $o_i[k]$ 指向量 o_i 在索引 k 处的值。

⁷这对应于 x_{i+1} 上的狄拉克 delta (Dirac delta) 分布与预测分布 $\text{softmax}(o_i)$ 之间的交叉熵。

4.2.1 在 PyTorch 中实现 SGD

为了实现我们的优化器,我们将继承 PyTorch 的 `torch.optim.Optimizer` 类。一个 `Optimizer` 子类必须实现两个方法:

`def __init__(self, params, ...)` 应当初始化你的优化器。这里, `params` 将是一个待优化参数的集合 (或者是参数组, `parameter groups`, 以使用户希望对模型的不同部分使用不同的超参数, 例如不同的学习率)。务必把 `params` 传给基类的 `__init__` 方法, 基类会存储这些参数以供 `step` 使用。你可以根据优化器的需要接收额外的参数 (例如学习率就是常见的一个), 并以字典的形式把它们传给基类构造函数, 其中键是你为这些参数选定的名称 (字符串)。

`def step(self)` 应当对参数执行一次更新。在训练循环中, 该方法会在反向传播 (backward pass) 之后被调用, 因此你可以访问到上一个批次上的梯度。该方法应当遍历每个参数张量 `p` 并就地 (in place) 修改它们, 即设置 `p.data`——它保存着与该参数关联的张量——其依据是梯度 `p.grad` (如果存在的话), 即损失相对于该参数的梯度张量。

PyTorch 的优化器 API 有一些细微之处, 因此用一个例子来解释会更容易。为了让我们的例子更丰富, 我们将实现一个 SGD 的小变体, 其中学习率会随训练衰减: 从初始学习率 α 开始, 随时间逐步采取越来越小的步长:

$$\theta_{t+1} = \theta_t - \frac{\alpha}{\sqrt{t+1}} \nabla L(\theta_t; B_t) \quad (20)$$

让我们看看这个版本的 SGD 如何作为一个 PyTorch `Optimizer` 来实现:

```
from collections.abc import Callable, Iterable
from typing import Optional
import torch
import math

class SGD(torch.optim.Optimizer):
    def __init__(self, params, lr=1e-3):
        if lr < 0:
            raise ValueError(f"Invalid learning rate: {lr}")
        defaults = {"lr": lr}
        super().__init__(params, defaults)

    def step(self, closure: Optional[Callable] = None):
        loss = None if closure is None else closure()
        for group in self.param_groups:
            lr = group["lr"] # Get the learning rate.
            for p in group["params"]:
                if p.grad is None:
                    continue

                state = self.state[p] # Get state associated with p.
                t = state.get("t", 0) # Get iteration number from the state, or 0.
                grad = p.grad.data # Get the gradient of loss with respect to p.
                p.data -= lr / math.sqrt(t + 1) * grad # Update weight tensor in-place.
```

```
state["t"] = t + 1 # Increment iteration number.  
  
return loss
```

在 `__init__` 中，我们把参数以及默认超参数传给基类构造函数（参数可能成组传入，每组有不同的超参数）。如果参数只是单个 `torch.nn.Parameter` 对象的集合，基类构造函数会创建一个单独的组，并为其分配默认超参数。然后在 `step` 中，我们遍历每个参数组，再遍历该组中的每个参数，并应用式 (20)。这里，我们把迭代次数作为与每个参数关联的状态保存：我们先读取该值，将其用于梯度更新，然后再更新它。

API 规定用户可能会传入一个可调用的闭包（closure），用于在优化器步进前重新计算损失。我们要用的优化器并不需要这一点，但为了符合 API，我们还是把它加上。

为了看到它的实际运行效果，我们可以使用下面这个最小的训练循环示例：

```
weights = torch.nn.Parameter(5 * torch.randn((10, 10)))  
opt = SGD([weights], lr=1)  
  
for t in range(100):  
    opt.zero_grad() # Reset the gradients for all learnable parameters.  
    loss = (weights**2).mean() # Compute a scalar loss value.  
    print(loss.cpu().item())  
  
    loss.backward() # Run backward pass, which computes gradients.  
    opt.step() # Run optimizer step.
```

这就是训练循环的典型结构：在每次迭代中，我们计算损失并运行优化器的一步。当训练语言模型时，我们的可学习参数将来自模型（在 PyTorch 中，`m.parameters()` 给出这个集合）。损失将在一个采样得到的数据批次上计算，但训练循环的基本结构是一样的。

题目 (learning_rate_tuning): 调节学习率 (1 分) 正如我们将要看到的，对训练影响最大的超参数之一就是学习率。让我们在这个玩具示例中实际看一看。用学习率的另外三个取值——`1e1`、`1e2` 和 `1e3`——运行上面的 SGD 示例，每次仅进行 10 次训练迭代。对于这些学习率，损失各自发生了什么？它衰减得更快、更慢，还是发散了（即在训练过程中不断增大）？

交付物：用一到两句话描述你观察到的行为。

4.3 AdamW

现代语言模型通常使用比 SGD 更复杂的优化器进行训练。近来使用的大多数优化器都是 Adam 优化器 [D. P. Kingma 等, 2015] 的衍生物。我们将使用 AdamW [I. Loshchilov 等, 2019]，它在近期工作中被广泛使用。AdamW 对 Adam 提出了一项改进：通过添加权重衰减（weight decay，在每次迭代中把参数拉向 0）来改善正则化，并且这种方式与梯度更新是解耦的（decoupled）。我们将按照 I. Loshchilov 等 [23] 中算法 2 的描述来实现 AdamW。

AdamW 是有状态的：对于每个参数，它都会维护其一阶矩和二阶矩的滑动估计（running estimate）。因此，AdamW 以额外的内存为代价，换取了更好的稳定性和收敛性。除了学习率 α 之外，AdamW 还有一对超参数 (β_1, β_2) ，用于控制对矩估计的更新，以及一个权重衰减率 λ 。典型应

用将 (β_1, β_2) 设为 $(0.9, 0.999)$ ，但像 LLaMA [H. Touvron 等, 2023] 和 GPT-3 [T. B. Brown 等, 2020] 这样的大型语言模型常常用 $(0.9, 0.95)$ 来训练。该算法可以写成如下形式，其中 ε 是一个小值（例如 10^{-8} ），用于在 v 中出现极小值时改善数值稳定性：

算法 1：AdamW 优化器

```

1  init( $\theta$ )                                ▷ 初始化可学习参数
2   $m \leftarrow 0$                             ▷ 一阶矩向量的初值；与  $\theta$  形状相同
3   $v \leftarrow 0$                             ▷ 二阶矩向量的初值；与  $\theta$  形状相同
4  for  $t = 1, \dots, T$  do
5    采样数据批次  $B_t$ 
6     $g \leftarrow \nabla_{\theta} \ell(\theta; B_t)$     ▷ 计算损失的梯度
7     $\alpha_t \leftarrow \alpha \frac{\sqrt{1 - \beta_2^t}}{1 - \beta_1^t}$     ▷ 计算第  $t$  次迭代调整后的  $\alpha$ 
8     $\theta \leftarrow \theta - \alpha \lambda \theta$         ▷ 应用权重衰减
9     $m \leftarrow \beta_1 m + (1 - \beta_1) g$         ▷ 更新一阶矩估计
10    $v \leftarrow \beta_2 v + (1 - \beta_2) g^2$       ▷ 更新二阶矩估计
11    $\theta \leftarrow \theta - \alpha_t \frac{m}{\sqrt{v} + \varepsilon}$     ▷ 应用经矩调整的权重更新
12 end for

```

算法 1：AdamW 优化器伪代码。

注意 t 从 1 开始。现在你将实现这个优化器。

题目 (adamw): 实现 AdamW (2 分) **交付物**: 将 AdamW 优化器实现为 `torch.optim.Optimizer` 的一个子类。你的类应在 `__init__` 中接收学习率 α ，以及 β 、 ε 和 λ 等超参数。为了帮助你保存状态，基类 `Optimizer` 提供了一个字典 `self.state`，它把 `nn.Parameter` 对象映射到一个字典，该字典存储你为该参数所需的任何信息（对 AdamW 而言，这就是矩估计）。实现 `[adapters.get_adamw_cls]` 并确保它通过 `uv run pytest -k test_adamw`。

题目 (adamw_accounting): 使用 AdamW 训练的资源核算 (2 分) 让我们计算一下运行 AdamW 需要多少内存和算力。假设我们对每个张量都使用 `float32`。

(a) 运行 AdamW 需要多少峰值内存 (peak memory)? 请根据参数、激活值 (activations)、梯度和优化器状态的内存使用情况, 对你的答案进行分解。用 `batch_size` 和模型超参数 (`vocab_size`、`context_length`、`num_layers`、`d_model`、`num_heads`) 来表示你的答案。假设 $d_{\text{ff}} = \frac{8}{3} \times d_{\text{model}}$ 。为简单起见，在计算激活值的内存使用时，只考虑以下几个组成部分：

- Transformer 块 (block)
 - RMSNorm (一个或多个)
 - 多头自注意力 (multi-head self-attention) 子层: QKV 投影、 QK^T 矩阵乘法、softmax、

对值的加权求和、输出投影。

- 逐位置前馈网络 (position-wise feed-forward, SwiGLU): W_1 、 W_2 、在门控分支 (gate branch) 上的 SiLU、逐元素乘积、 W_3

4.4 学习率调度 (Learning rate scheduling)

能让损失下降最快的学习率 (learning rate) 取值, 在训练过程中往往是变化的。在训练 Transformer 时, 一种典型做法是使用**学习率调度** (learning rate schedule): 开始时使用较大的学习率, 在前期做出更快的参数更新, 随着模型训练再缓慢衰减到较小的取值。⁸在本作业中, 我们将实现用于训练 LLaMA 的余弦退火 (cosine annealing) 调度 [H. Touvron 等, 2023]。

一个调度器 (scheduler) 本质上就是一个函数: 它接收当前步数 t 以及其他相关参数 (例如初始学习率和最终学习率), 并返回在第 t 步用于梯度更新的学习率。最简单的调度是常数函数, 无论给定任何 t 都返回相同的学习率。

余弦退火学习率调度接收以下输入: (i) 当前迭代步数 t , (ii) 最大学习率 $\alpha_{\max}$, (iii) 最小 (最终) 学习率 $\alpha_{\min}$, (iv) 预热 (warm-up) 迭代步数 T_w , 以及 (v) 余弦退火的最终迭代步数 T_c 。第 t 次迭代的学习率定义如下:

(预热) 若 $t < T_w$, 则

$$\alpha_t = \frac{t}{T_w} \alpha_{\max}.$$

(余弦退火) 若 $T_w \leq t \leq T_c$, 则

$$\alpha_t = \alpha_{\min} + \frac{1}{2} \left(1 + \cos \left(\frac{t - T_w}{T_c - T_w} \pi \right) \right) (\alpha_{\max} - \alpha_{\min}).$$

(退火后) 若 $t > T_c$, 则

$$\alpha_t = \alpha_{\min}.$$

题目 (learning_rate_schedule): 实现带预热的余弦学习率调度 (1 分) 编写一个函数, 它接收 t 、 $\alpha_{\max}$ 、 $\alpha_{\min}$ 、 T_w 和 T_c , 并按上面定义的调度器返回学习率 α_t 。然后实现 `adapters.get_lr_cosine_schedule`, 并确保它通过 `uv run pytest -k test_get_lr_cosine_schedule`。

4.5 梯度裁剪 (Gradient clipping)

在训练过程中, 我们有时会遇到产生很大梯度的训练样本, 这会使训练变得不稳定。为缓解这一问题, 实践中常用的一种技术是**梯度裁剪** (gradient clipping)。其思想是: 在每次反向传播 (backward pass) 之后、执行优化器步进之前, 对梯度的范数施加一个上限。

给定 (所有参数的) 梯度 g , 我们计算它的 ℓ_2 范数 $\|g\|_2$ 。如果该范数小于某个最大值 M , 则保持 g 不变; 否则将 g 按因子 $\frac{M}{\|g\|_2 + \varepsilon}$ 缩小 (其中加上一个很小的 ε , 如 10^{-6} , 是为了数值稳定性)。注意, 缩放后得到的范数会略小于 M 。

⁸有时也常用一种学习率会回升 (重启, restart) 的调度, 以帮助跳出局部极小值。

题目 (gradient_clipping): 实现梯度裁剪 (1 分) 编写一个实现梯度裁剪的函数。你的函数应接收一个参数列表以及一个最大 ℓ_2 范数。它应原地 (in place) 修改每个参数的梯度。使用 $\epsilon = 10^{-6}$ (PyTorch 的默认值)。然后实现适配器 `adapters.run_gradient_clipping`, 并确保它通过 `uv run pytest -k test_gradient_clipping`。

5 训练循环 (Training loop)

现在我们终于要把目前为止构建的几个主要组件组装到一起：分词后的数据、模型，以及优化器。

5.1 数据加载器 (Data Loader)

分词后的数据（例如你在 `tokenizer_experiments` 中准备的数据）是单一的 token 序列 $x = (x_1, \dots, x_n)$ 。尽管源数据可能由若干独立文档组成（例如不同的网页，或源代码文件），一种常见做法是把所有这些文档拼接成单一的 token 序列，并在它们之间加入分隔符（例如 `<|endoftext|>` token）。

数据加载器 (data loader) 会把它转化为一连串的批次 (batch)，其中每个批次由 B 条长度为 m 的序列组成，并配以对应的“下一个 token”，长度同样为 m 。例如，当 $B = 1, m = 3$ 时， $([x_2, x_3, x_4], [x_3, x_4, x_5])$ 就是一个可能的批次。

以这种方式加载数据出于若干原因简化了训练。首先，任意 $1 \leq i \leq n - m$ 都给出一条有效的训练序列，因此采样训练序列十分简单。由于所有训练序列都具有相同长度，无需对输入序列做填充 (pad)，这提高了硬件利用率（同时也通过增大批大小 B 来提高利用率）。最后，我们也无需加载整个数据集就能采样训练数据，这使得处理那些原本无法装入内存的大型数据集变得容易。

题目 (data_loading): 实现数据加载 (2 分) **交付物:** 编写一个函数，它接收一个 numpy 数组 x (包含 token ID 的整数数组)、一个 `batch_size`、一个 `context_length`，以及一个 PyTorch 设备字符串 (例如 `'cpu'` 或 `'cuda:0'`)，并返回一对张量：采样得到的输入序列，以及对应的“下一个 token”目标。两个张量都应具有形状 $(\text{batch_size}, \text{context_length})$ ，包含 token ID，且都应放置在所请求的设备上。为了用我们提供的测试来检验你的实现，你首先需要实现 `adapters.run_get_batch` 处的测试适配器。然后运行 `uv run pytest -k test_get_batch` 来测试你的实现。

低资源提示：在 CPU 或 Apple Silicon 上加载数据 如果你打算在 CPU 或 Apple Silicon 上训练你的语言模型 (LM)，你需要把数据移动到正确的设备上（类似地，之后你的模型也应使用相同的设备）。

如果你在 CPU 上，可以使用设备字符串 `'cpu'`；在 Apple Silicon (M* 系列芯片) 上，可以使用设备字符串 `'mps'`。

关于 MPS 的更多信息，请查阅以下资源：

- <https://docs.pytorch.org/docs/stable/mps.html>
- <https://docs.pytorch.org/docs/stable/notes/mps.html>

- <https://developer.apple.com/documentation/metalperformanceshaders>

如果数据集太大无法装入内存怎么办？我们可以使用一个名为 `mmap` 的 Unix 系统调用，它把磁盘上的文件映射到虚拟内存，并在访问到相应内存位置时惰性地（lazily）加载文件内容。这样，你就可以“假装”整个数据集都在内存中。Numpy 通过 `np.memmap` 实现这一点（如果你最初是用 `np.save` 保存数组的，则也可对 `np.load` 使用标志 `mmap_mode='r'`），它会返回一个类 numpy 数组的对象，在你访问条目时按需加载。在训练期间从你的数据集（即一个 numpy 数组）采样时，务必以内存映射模式加载数据集（视你保存数组的方式而定，使用 `np.memmap` 或对 `np.load` 使用标志 `mmap_mode='r'`）。还要确保你指定的 `dtype` 与你所加载的数组相匹配。明确地验证内存映射数据看起来正确可能会有帮助（例如，不包含超出预期词表大小的取值）。

5.2 检查点 (Checkpointing)

除了加载数据，我们在训练时还需要保存模型。运行作业时，我们经常希望能够恢复一次中途停止的训练（例如，由于作业超时、机器故障等）。即使一切顺利，我们之后也可能希望访问中间模型（例如，事后研究训练动态、在训练的不同阶段从模型中采样等）。

一个检查点 (checkpoint) 应包含我们恢复训练所需的全部状态。我们当然至少希望能够恢复模型权重。如果使用有状态的优化器（如 AdamW），我们还需要保存优化器的状态（例如，对于 AdamW，即各阶矩估计）。最后，为了恢复学习率调度，我们需要知道停止时所处的迭代步数。PyTorch 让保存这些都很简单：每个 `nn.Module` 都有一个 `state_dict()` 方法，返回一个包含全部可学习权重的字典；之后我们可用配套的 `load_state_dict()` 方法恢复这些权重。任何 `torch.optim.Optimizer` 也同样如此。最后，`torch.save(obj, dest)` 可以把一个对象（例如，一个以张量为部分取值、但也包含像整数这样的普通 Python 对象的字典）转储到文件（路径）或类文件对象，之后可用 `torch.load(src)` 把它加载回内存。

题目 (checkpointing): 实现模型检查点 (1 分) 实现以下两个用于加载和保存检查点的函数：
`def save_checkpoint(model, optimizer, iteration, out)` 应把来自 `model`、`optimizer` 和 `iteration` 的全部状态转储到类文件对象 `out` 中。你可以使用 `model` 和 `optimizer` 二者的 `state_dict` 方法获取它们的相关状态，并用 `torch.save(obj, out)` 把 `obj` 转储到 `out` (PyTorch 在此处支持路径或类文件对象)。一种典型选择是让 `obj` 是一个字典，但只要你能加载你的检查点，你可以使用任何你想要的格式。
该函数期望以下参数：

- `model: torch.nn.Module`
- `optimizer: torch.optim.Optimizer`
- `iteration: int`
- `out: str | os.PathLike | typing.BinaryIO | typing.IO[bytes]`

`def load_checkpoint(src, model, optimizer)` 应从 `src` (路径或类文件对象) 加载一个检查点，然后从该检查点恢复 `model` 和 `optimizer` 的状态。你的函数应返回保存到检查点中的迭

代步数。你可以使用 `torch.load(src)` 恢复你在 `save_checkpoint` 实现中所保存的内容，并使用 `model` 和 `optimizer` 二者的 `load_state_dict` 方法把它们恢复到之前的状态。

该函数期望以下参数：

- `src: str | os.PathLike | typing.BinaryIO | typing.IO[bytes]`
- `model: torch.nn.Module`
- `optimizer: torch.optim.Optimizer`

实现 `adapters.run_save_checkpoint` 和 `adapters.run_load_checkpoint` 适配器，并确保它们通过 `uv run pytest -k test_checkpointing`。

5.3 训练循环 (Training loop)

现在，终于到了把你实现的所有组件组装进你的主训练脚本的时候了。让脚本能够方便地用不同超参数 (hyperparameter) 启动训练（例如，把它们作为命令行参数接收）会很有回报，因为之后你将多次这样做，以研究不同选择如何影响训练。

题目 (training_together): 组装到一起 (4 分) **交付物:** 编写一个脚本，运行一个训练循环，在用户提供的输入上训练你的模型。具体而言，我们建议你的训练脚本（至少）允许以下功能：

- 能够配置和控制各种模型与优化器的超参数。
- 用 `np.memmap` 高效地、节省内存地加载大型训练集和验证集。
- 把检查点序列化到用户提供的路径。
- 周期性地记录训练和验证表现（例如，记录到控制台和/或像 Weights and Biases 这样的外部服务^a）。

^a<https://wandb.ai>

6 生成文本 (Generating text)

既然我们已经能够训练模型，所需的最后一块拼图就是从模型生成文本的能力。回想一下，语言模型接收一个(可能成批的)长度为 `sequence_length` 的整数序列，并产生一个大小为 `(sequence_length, vocab_size)` 的矩阵，其中序列的每个元素都是一个概率分布，预测该位置之后的下一个 token。我们现在将编写几个函数，把它转化为一种为新序列采样的方案。

Softmax

按照标准约定，语言模型的输出是最终线性层的输出 (“logits”)，因此我们必须通过 softmax 运算把它转化为归一化的概率，这一运算我们在前面的式 (10) 中已经见过。

解码 (Decoding)

为了从我们的模型生成文本（解码，decode），我们将给模型提供一段前缀 token 序列（“提示”，prompt），并要求它产生一个在词表上的概率分布，用以预测序列中的下一个 token。然后，我们从这个在词表项上的分布中采样，以确定下一个输出 token。

具体而言，解码过程的一步应接收一个序列 $x_{1...t}$ ，并通过以下方程返回一个 token x_{t+1} ：

$$P(x_{t+1} = i \mid x_{1...t}) = \frac{\exp(v_i)}{\sum_j \exp(v_j)} \quad (21)$$

$$v = \text{TransformerLM}(x_{1...t})_t \in \mathbb{R}^{\text{vocab_size}} \quad (22)$$

其中 TransformerLM 是我们的模型，它接收一个长度为 `sequence_length` 的序列作为输入，并产生一个大小为 $(\text{sequence_length}, \text{vocab_size})$ 的矩阵；我们取这个矩阵的最后一个元素，因为我们要的是第 t 个位置上对下一个 token 的预测。

通过反复地从这些一步条件分布中采样（把先前生成的输出 token 追加到下一个解码时间步的输入中），直到我们生成序列结束 token `<|endoftext|>`（或用户指定的最大生成 token 数），这就给了我们一个基本的解码器。

解码技巧 (Decoder tricks)

我们将用小模型做实验，而小模型有时会生成质量很低的文本。两个简单的解码技巧有助于修正这些问题。首先，在**温度缩放**（temperature scaling）中，我们用一个温度参数 τ 来修改 softmax，新的 softmax 为

$$\text{softmax}(v, \tau)_i = \frac{\exp(v_i/\tau)}{\sum_{j=1}^{\text{vocab_size}} \exp(v_j/\tau)}. \quad (23)$$

注意，令 $\tau \rightarrow 0$ 会使得 v 中最大的元素占据主导，softmax 的输出变成集中于这个最大元素的独热（one-hot）向量。

第二个技巧是核采样（nucleus sampling），也称 top- p 采样，我们通过截断低概率词元（token）来修改采样分布。设 q 是我们从一个（经温度缩放的）softmax 得到的概率分布，其大小为 `vocab_size`。带超参数 p 的核采样按下式产生下一个词元：

$$P(x_{t+1} = i \mid q) = \begin{cases} \frac{q_i}{\sum_{j \in V(p)} q_j} & \text{若 } i \in V(p) \\ 0 & \text{其他情况} \end{cases} \quad (24)$$

其中 $V(p)$ 是满足 $\sum_{j \in V(p)} q_j \geq p$ 的最小索引集合。你可以很容易地计算这个量：先按幅值对概率分布 q 排序，然后从最大的词表元素开始依次选取，直到累计达到目标水平 p 。

题目 (decoding)：解码 (3 分) 交付物：实现一个从你的语言模型进行解码的函数。我们建议你支持以下特性：

- 为用户提供的提示词（prompt）生成补全（即输入某段 $x_{1...t}$ ，并持续采样补全，直到遇到 `<|endoftext|>` 词元）。
- 允许用户控制生成词元的最大数量。
- 给定一个期望的温度（temperature）值，在采样之前对预测的下一词元分布应用 softmax

温度缩放。

- Top- p 采样（见 A. Holtzman 等人，2020，也称核采样），由用户指定一个阈值。

7 实验

现在是时候把一切组合起来，在预训练数据集上训练（小型）语言模型了。

7.1 如何运行实验与交付物

理解 Transformer 各架构组件背后原理的最佳方式，就是亲自动手修改并运行它。没有什么能替代亲身实践。

为此，能够快速、一致地做实验并记录你的所作所为非常重要。为了快速实验，我们将在一个小规模模型（约 1700 万个总参数）和一个简单数据集（TinyStories）上运行许多实验。为了保持一致性，你将以系统化的方式对组件进行消融（ablation）并改变超参数；而为了保留记录，我们会要求你提交一份实验日志以及与每个实验相关联的学习曲线（learning curve）。

为了能够提交损失曲线，请确保周期性地评估验证损失（validation loss），并同时记录步数和挂钟时间（wall-clock time）。你可能会发现像 Weights and Biases 这样的日志基础设施很有帮助。

题目 (experiment_log): 实验日志记录（3 分）为你的训练与评估代码创建实验追踪基础设施，使你能够相对于梯度步数和挂钟时间来追踪你的实验和损失曲线。

交付物: 为你的实验提供日志基础设施代码，以及一份实验日志（记录你为本节后续作业题目所尝试一切的文档）。

7.2 TinyStories

我们将从一个非常简单的数据集（TinyStories；R. Eldan 等人，[1]）开始，在该数据集上模型训练得很快，并且我们能看到一些有趣的行为。获取该数据集的说明见第 1 节。该数据集的样子示例如下。

示例 (tinystories_example): TinyStories 中的一个例子 Once upon a time there was a little boy named Ben. Ben loved to explore the world around him. He saw many amazing things, like beautiful vases that were on display in a store. One day, Ben was walking through the store when he came across a very special vase. When Ben saw it he was amazed! He said, “Wow, that is a really amazing vase! Can I buy it?” The shopkeeper smiled and said, “Of course you can. You can take it home and show all your friends how amazing it is!” So Ben took the vase home and he was so proud of it! He called his friends over and showed them the amazing vase. All his friends thought the vase was beautiful and couldn’t believe how lucky Ben was. And that’s how Ben found an amazing vase in the store!

7.2.1 超参调优

我们会告诉你一些非常基础的超参数作为起点，并要求你为另一些超参数找到效果不错的设置。

词表大小 (Vocab size) 10000。典型的词表大小在数万到数十万之间。你应当改变这个值，看看词表和模型行为如何变化。

上下文长度 (Context length) 256。像 TinyStories 这样简单的数据集可能不需要很长的序列长度，但对于后面的 OpenWebText 数据，你可能会想要改变它。尝试改变它，观察它对每次迭代运行时间和最终困惑度 (perplexity) 两方面的影响。

d_{model} 512。这比许多小型 Transformer 论文中所用的 768 维略小，但这会让运行更快。

d_{ff} 1344。这大约是 $\frac{8}{3} d_{\text{model}}$ ，同时又是 64 的倍数，这对 GPU 性能有好处。

RoPE 的 theta 参数 Θ 10000。

层数与头数 (Number of layers and heads) 4 层, 16 个头。二者合起来将给出约 1700 万个非嵌入 (non-embedding) 参数，这是一个相当小的 Transformer。

处理的总词元数 (Total tokens processed) 327,680,000 (你的批大小 $\times$ 总步数 $\times$ 上下文长度应当大致等于这个值)。

你应当通过一些试错来为以下其他超参数找到好的默认值：学习率 (learning rate)、学习率预热 (learning rate warmup)、其他 AdamW 超参数 (β_1 、 β_2 、 ϵ) 以及权重衰减 (weight decay)。你可以在 D. P. Kingma 等人 [22] 中找到这些超参数的一些典型取值。

7.2.2 组合实现

现在你可以把一切组合起来：获取一个训练好的 BPE 分词器 (tokenizer)，对训练数据集分词，并在你所编写的训练循环中运行它。**重要提示：**如果你的实现正确且高效，上述超参数在 1 块 B200 GPU 上应当导致大约 20–30 分钟的运行时间。如果你的运行时间长得多，请检查并确保你的数据加载、检查点 (checkpointing) 或验证损失代码没有成为运行时间的瓶颈，并确保你的实现进行了恰当的批处理。

7.2.3 调试模型架构的技巧

我们强烈建议你熟练使用 IDE 内置的调试器 (例如 VSCode/Zed)，与用打印语句调试相比，这会为你节省时间。如果你使用文本编辑器，可以使用类似 ipdb 的工具。调试模型架构时的其他一些良好实践包括：

- 开发任何神经网络架构时，常见的第一步是过拟合 (overfit) 到单个小批量 (minibatch)。如果你的实现正确，你应当能够迅速把训练损失驱动到接近零。
- 在各个模型组件中设置调试断点，并检查中间张量的形状，以确保它们符合你的预期。
- 监控激活值、模型权重和梯度的范数 (norm)，以确保它们没有爆炸或消失。

题目 (learning_rate): 调优学习率 (2 B200 小时) (3 分) 学习率是最重要的待调超参数之一。以你已经训练好的基础模型为出发点，回答以下问题：

(a) 在学习率上进行一次超参数扫描 (sweep)，并报告最终损失 (如果优化器发散，则注明发

散)。

交付物：与多个学习率相关联的学习曲线。解释你的超参数搜索策略。

交付物：一个在 TinyStories 上（按每词元计）验证损失至多为 1.45 的模型。

低资源提示：在 CPU 或 Apple Silicon 上训练若干步 如果你在 `cpu` 或 `mps` 上运行，你应当转而把处理的总词元数减少到 40,000,000，这足以产生相当流畅的文本。你也可以把目标验证损失从 1.45 提高到 2.00。

在一块 M4 Max 芯片和 36 GB 内存上运行我们的参考解答代码（已调好学习率），我们使用批大小 $\times$ 总步数 $\times$ 上下文长度 = $32 \times 5000 \times 256 = 40,960,000$ 个词元，这在 `cpu` 上需要 1 小时 22 分钟，在 `mps` 上需要 36 分钟。在第 5000 步时，我们达到 1.80 的验证损失。一些附加提示：

- 当使用 N 个训练步时，我们建议调整余弦学习率衰减（cosine learning rate decay）调度，使其衰减恰好在第 N 步结束（即达到最小学习率）。
- 当使用 `mps` 时，不要使用 TF32 内核，即不要设置

```
torch.set_float32_matmul_precision('high')
```

（在 `cuda` 设备上你可能会这样做）。我们曾尝试在 `mps` 上启用 TF32 内核（torch 版本 2.9.0），发现该后端有时会无声地使用有缺陷的内核，从而导致训练不稳定。

- 你可以通过用 `torch.compile` 对模型进行即时（JIT）编译来加速训练。具体而言：
 - 在 `cpu` 上，用以下方式编译你的模型

```
model = torch.compile(model)
```

- 在 `mps` 上，你可以这样对反向传播做一定优化

```
model = torch.compile(model, backend="aot_eager")
```

截至 torch 版本 2.9.0，`mps` 上尚不支持使用 Inductor 进行编译。

(b) 民间经验是说最佳学习率处于“稳定性的边缘（edge of stability）”。研究一下学习率开始发散的那个点与你的最佳学习率之间有何关系。

交付物：学习率逐渐增大的学习曲线（其中至少包含一次发散的运行），以及关于这与收敛速率有何关系的分析。

现在让我们改变批大小（batch size），看看训练会发生什么。批大小很重要——它们让我们通过做更大的矩阵乘法从 GPU 获得更高的效率，但我们是否总是希望批大小越大越好呢？让我们运行一些实验来一探究竟。

题目 (batch_size_experiment): 批大小变化 (1 B200 小时) (1 分) 把你的批大小从 1 一直变化到 GPU 显存上限。至少尝试中间的几个批大小, 包括 64 和 128 这样的典型大小。

交付物: 不同批大小各次运行的学习曲线。如有必要应当再次优化学习率。

交付物: 用几句话讨论你关于批大小及其对训练影响的发现。

有了你的解码器, 现在我们可以生成文本了! 我们将从模型生成, 并看看效果如何。作为参考, 你得到的输出应当至少和下面的例子一样好。

示例 (ts_generate_example): 一个 TinyStories 语言模型的样例输出 Once upon a time, there was a pretty girl named Lily. She loved to eat gum, especially the big black one. One day, Lily's mom asked her to help cook dinner. Lily was so excited! She loved to help her mom. Lily's mom made a big pot of soup for dinner. Lily was so happy and said, "Thank you, Mommy! I love you." She helped her mom pour the soup into a big bowl. After dinner, Lily's mom made some yummy soup. Lily loved it! She said, "Thank you, Mommy! This soup is so yummy!" Her mom smiled and said, "I'm glad you like it, Lily." They finished cooking and continued to cook together. The end.

低资源提示: 在 CPU 或 Apple Silicon 上生成文本 如果你转而使用处理 40M 词元的低资源配置, 你应当看到生成的文本仍然像是英语, 但不如上面那样流畅。例如, 下面是我们在 40M 词元上训练的 TinyStories 语言模型的样例输出:

Once upon a time, there was a little girl named Sue. Sue had a tooth that she loved very much. It was his best head. One day, Sue went for a walk and met a ladybug! They became good friends and played on the path together.

"Hey, Polly! Let's go out!" said Tim. Sue looked at the sky and saw that it was difficult to find a way to dance shining. She smiled and agreed to help the talkingi'

As Sue watched the sky moved, what it was. She

下面是精确的题目陈述以及我们的要求:

题目 (generate): 生成文本 (1 分) 使用你的解码器和训练好的检查点, 报告你的模型生成的文本。你可能需要调整解码器参数 (温度、top- p 等) 以获得流畅的输出。

交付物: 至少 256 个词元的文本转储 (或直到第一个 `<|endoftext|>` 词元为止), 并对该输出的流畅度作简要评论, 以及至少两个影响该输出好坏的因素。

7.3 消融与架构修改

理解 Transformer 的最佳方式就是亲自修改它并观察它的行为。下面我们将做几个简单的消融与修改。

消融 1：层归一化 (layer normalization) 人们常说层归一化对 Transformer 训练的稳定性很重要。但也许我们想冒险一把。让我们从每个 Transformer 块中移除 RMSNorm，看看会发生什么。

题目 (layer_norm_ablation): 移除 RMSNorm 并训练 (0.5 B200 小时) (1 分) 从你的 Transformer 中移除所有的 RMSNorm 并训练。在先前的最优学习率下会发生什么？你能否通过使用更低的学习率来获得稳定性？

交付物：移除 RMSNorm 并训练时的一条学习曲线，以及在最佳学习率下的一条学习曲线。

交付物：用几句话评论 RMSNorm 的影响。

现在让我们研究另一种乍看上去似乎很随意的层归一化选择。前置归一化 (pre-norm) 的 Transformer 块定义为

$$z = x + \text{MultiHeadSelfAttention}(\text{RMSNorm}(x)) \quad (25)$$

$$y = z + \text{FFN}(\text{RMSNorm}(z)). \quad (26)$$

这是对原始 Transformer 架构为数不多的「共识」修改之一。原始 Transformer 采用的是后置归一化 (post-norm) 方式：

$$z = \text{RMSNorm}(x + \text{MultiHeadSelfAttention}(x)) \quad (27)$$

$$y = \text{RMSNorm}(z + \text{FFN}(z)). \quad (28)$$

让我们退回到后置归一化的方式，看看会发生什么。

题目 (pre_norm_ablation): 实现 post-norm 并训练 (0.5 B200 小时) (1 分) 把你的前置归一化 (pre-norm) Transformer 实现改成后置归一化 (post-norm)。用 post-norm 模型训练，看看会发生什么。

交付物：一条 post-norm Transformer 的学习曲线，并与 pre-norm 的曲线对比。

我们会看到，层归一化 (layer normalization) 对 Transformer 的行为有重大影响，甚至连层归一化所放置的位置都很重要。

消融 2：位置编码 (position embeddings) 接下来我们研究位置编码对模型性能的影响。具体来说，我们会把基线模型（使用 RoPE）与完全不使用任何位置编码 (NoPE, No Position Embedding) 进行对比。事实证明，仅含解码器 (decoder-only) 的 Transformer——也就是像我们所实现的、带因果掩码 (causal mask) 的模型——在理论上即使不显式提供位置编码，也能推断出相对或绝对的位置信息（见 Y.-H. H. Tsai 等人，2019；A. Kazemnejad 等人，2023）。现在我们就用实验来检验 NoPE 相比 RoPE 的表现如何。

题目 (no_pos_emb): 实现 NoPE (0.5 B200 小时) (1 分) 把你带 RoPE 的 Transformer 实现修改为完全移除位置编码信息，看看会发生什么。

交付物：一条对比 RoPE 与 NoPE 性能的学习曲线。

消融 3: SwiGLU vs. SiLU 接下来，我们沿用 N. Shazeer (2020) 的做法，通过对比 SwiGLU 前馈网络与「使用 SiLU 激活但不带门控线性单元 (GLU, Gated Linear Unit)」的前馈网络，来检验门控 (gating) 在前馈网络中的重要性：

$$\text{FFN}_{\text{SiLU}}(x) = W_2 \text{SiLU}(W_1 x). \quad (29)$$

回忆一下，在我们的 SwiGLU 实现中，内部前馈层的维度大约设为 $d_{\text{ff}} = \frac{8}{3} d_{\text{model}}$ （同时保证 $d_{\text{ff}} \bmod 64 = 0$ ，以便利用 GPU 的张量核心 tensor core）。在本次消融的基线中，你的 FFN_{SiLU} 实现应改为设 $d_{\text{ff}} = 4 \times d_{\text{model}}$ ，以大致匹配默认 SwiGLU 前馈网络的参数量（后者有三个而非两个权重矩阵）。

题目 (swiglu_ablation): SwiGLU vs. SiLU (0.5 B200 小时) (1 分)

交付物：一条在参数量大致匹配的前提下、对比 SwiGLU 与 SiLU 前馈网络性能的学习曲线。

交付物：用几句话讨论你的发现。

低资源提示：GPU 资源有限的在线学生应在 TinyStories 上测试改动 在本作业接下来的部分，我们会转向一个规模更大、噪声更多的网络数据集 (OpenWebText)，来实验各种架构修改，并（可选地）向课程排行榜提交结果。

在 OpenWebText 上把语言模型训练到流畅需要很长时间，因此我们建议 GPU 资源有限的在线学生继续在 TinyStories 上测试改动（用验证损失作为评估性能的指标）。

7.4 在 OpenWebText 上运行

现在我们转向一个更标准的预训练数据集，它由网络爬取 (web crawl) 构建而成。我们同样提供了 OpenWebText (A. Gokaslan 等人, 2019) 的一个小型样本，作为单个文本文件给出：获取该方法见第 1 节。

下面是 OpenWebText 中的一个例子。请注意其中的文本要真实、复杂、多样得多。你或许会想浏览一下训练数据集，以体会网络爬取语料的训练数据是什么样子。

示例 (owt_example): 来自 OWT 的一个样例 Baseball Prospectus director of technology Harry Pavlidis took a risk when he hired Jonathan Judge.

Pavlidis knew that, as Alan Schwarz wrote in *The Numbers Game*, “no corner of American culture is more precisely counted, more passionately quantified, than performances of baseball players.” With a few clicks here and there, you can find out that Noah Syndergaard’s fastball revolves more than 2,100 times per minute on its way to the plate ...The rising ocean of data has empowered an increasingly important actor in baseball’s culture: the analytical hobbyist.

...Pavlidis had a hunch that Judge “got it” based on the latter’s writing and their interaction at a

site-sponsored ballpark event. [...]

注意：对于本实验，你可能需要重新调整诸如学习率或批大小这样的超参数。

题目 (main_experiment)：在 OWT 上做实验（2 B200 小时）（2 分）用与 TinyStories 相同的模型架构和相同的总训练迭代次数，在 OpenWebText 上训练你的语言模型。这个模型表现如何？

交付物：你的语言模型在 OpenWebText 上的学习曲线。描述与 TinyStories 相比损失的差异——我们应如何解读这些损失？

交付物：用与 TinyStories 输出相同的格式，给出 OpenWebText 语言模型生成的文本。这段文本的流畅度如何？为什么即使我们用了与 TinyStories 相同的模型和算力预算，输出质量却更差？

7.5 你自己的改动 + 排行榜

恭喜你走到了这一步，你就快完成了！现在你将尝试改进 Transformer 架构，看看你的超参数和架构与班上其他同学相比表现如何。

排行榜规则 除以下限制外没有其他约束：

- **运行时间 (Runtime)：**你的提交在一块 B200 上最多运行 45 分钟。如果你使用 SLURM 或 Modal，建议在提交脚本中强制这一限制。
- **数据 (Data)：**你只能使用我们提供的 OpenWebText 训练数据集。

除此之外，你可以随心所欲地尝试任何想法。

如果你需要一些实现思路，可以参考以下资源：

- 最先进的开源大语言模型系列，例如 Llama 3 (A. Grattafiori 等人, 2024) 或 Qwen 2.5 (A. Yang 等人, 2024)。
- NanoGPT speedrun 仓库 (github.com/KellerJordan/modded-nanogpt)，社区成员在此发布了许多用于「竞速 (speedrunning)」小规模语言模型预训练的有趣改动。例如，一个可追溯到原始 Transformer 论文的常见改动，是把输入嵌入与输出嵌入的权重绑定 (weight tying) 在一起 (见 A. Vaswani 等人 [8] 第 3.4 节，以及 A. Chowdhery 等人 [16] 第 2 节)。如果你尝试权重绑定，可能需要减小嵌入/语言模型头初始化的标准差。

在尝试完整的 45 分钟运行之前，你会想先在 OpenWebText 的一个小子集或在 TinyStories 上测试这些改动。

需要提醒的是，你在本排行榜中发现「效果不错」的某些改动，未必能推广到更大规模的预训练。我们会在课程的「缩放定律 (scaling laws)」单元中进一步探讨这一点。

题目 (leaderboard): 排行榜 (10 B200 小时) (6 分) 你将在上述排行榜规则下训练一个模型, 目标是在 0.75 B200 小时之内最小化语言模型的验证损失。

交付物: 记录到的最终验证损失、一条清晰地以「墙钟时间 (wall-clock time) 少于 45 分钟」为横轴的学习曲线, 以及一段你所做工作的说明。我们期望排行榜提交至少要击败 5.0 损失这个朴素基线。提交到此处的排行榜: github.com/stanford-cs336/assignment1-basics-leaderboard。

参考文献

- [1] R. Eldan and Y. Li, “TinyStories: How Small Can Language Models Be and Still Speak Coherent English?.” 2023.
- [2] A. Gokaslan, V. Cohen, E. Pavlick, and S. Tellex, “OpenWebText corpus.” 2019.
- [3] R. Sennrich, B. Haddow, and A. Birch, “Neural Machine Translation of Rare Words with Subword Units,” in Proc. of ACL, 2016.
- [4] C. Wang, K. Cho, and J. Gu, “Neural Machine Translation with Byte-Level Subwords.” 2019.
- [5] P. Gage, “A new algorithm for data compression,” C Users Journal, vol. 12, no. 2, pp. 23–38, Feb. 1994.
- [6] A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, and I. Sutskever, “Language Models are Unsupervised Multitask Learners.” 2019.
- [7] A. Radford, K. Narasimhan, T. Salimans, and I. Sutskever, “Improving Language Understanding by Generative Pre-Training.” 2018.
- [8] A. Vaswani et al., “Attention is All you Need,” in Proc. of NeurIPS, 2017.
- [9] T. Q. Nguyen and J. Salazar, “Transformers without Tears: Improving the Normalization of Self-Attention,” in Proc. of IWSLT, 2019.
- [10] R. Xiong et al., “On Layer Normalization in the Transformer Architecture,” in Proc. of ICML, 2020.
- [11] J. L. Ba, J. R. Kiros, and G. E. Hinton, “Layer Normalization.” 2016.
- [12] H. Touvron et al., “LLaMA: Open and Efficient Foundation Language Models.” 2023.
- [13] B. Zhang and R. Sennrich, “Root Mean Square Layer Normalization,” in Proc. of NeurIPS, 2019.
- [14] A. Grattafiori et al., “The Llama 3 Herd of Models.” <https://arxiv.org/abs/2407.21783>
- [15] A. Yang et al., “Qwen2.5 Technical Report,” arXiv preprint arXiv:2412.15115, 2024.
- [16] A. Chowdhery et al., “PaLM: Scaling Language Modeling with Pathways.” 2022.
- [17] D. Hendrycks and K. Gimpel, “Bridging Nonlinearities and Stochastic Regularizers with Gaussian Error Linear Units.” 2016.
- [18] S. Elfving, E. Uchibe, and K. Doya, “Sigmoid-Weighted Linear Units for Neural Network Function Approximation in Reinforcement Learning.” <https://arxiv.org/abs/1702.03118>
- [19] Y. N. Dauphin, A. Fan, M. Auli, and D. Grangier, “Language Modeling with Gated Convolutional Networks.” <https://arxiv.org/abs/1612.08083>
- [20] N. Shazeer, “GLU Variants Improve Transformer.” 2020.

- [21] J. Su, Y. Lu, S. Pan, B. Wen, and Y. Liu, “RoFormer: Enhanced Transformer with Rotary Position Embedding.” 2021.
- [22] D. P. Kingma and J. Ba, “Adam: A Method for Stochastic Optimization,” in Proc. of ICLR, 2015.
- [23] I. Loshchilov and F. Hutter, “Decoupled Weight Decay Regularization,” in Proc. of ICLR, 2019.
- [24] T. B. Brown et al., “Language Models are Few-Shot Learners,” in Proc. of NeurIPS, 2020.
- [25] J. Kaplan et al., “Scaling Laws for Neural Language Models.” 2020.
- [26] J. Hoffmann et al., “Training Compute-Optimal Large Language Models.” 2022.
- [27] A. Holtzman, J. Buys, L. Du, M. Forbes, and Y. Choi, “The Curious Case of Neural Text Degeneration,” in Proc. of ICLR, 2020.
- [28] Y.-H. H. Tsai, S. Bai, M. Yamada, L.-P. Morency, and R. Salakhutdinov, “Transformer Dissection: An Unified Understanding for Transformer’s Attention via the Lens of Kernel,” in Proc. of EMNLP-IJCNLP, 2019, pp. 4344–4353.
- [29] A. Kazemnejad, I. Padhi, K. Natesan, P. Das, and S. Reddy, “The Impact of Positional Encoding on Length Generalization in Transformers,” in NeurIPS, 2023.